

kayfabe

A Mode-2 NVIDIA GPU forwarding stack for KVM/QEMU guests

An architecture whitepaper, written to be attacked

Subject. Two repositories on one machine. `/workspace/nvidia-gpu-passthrough` — the C research artifact, 739 commits, 44 031 lines of C, developed 2026-05-24 to 2026-07-24; it works on real hardware. `/workspace/nvkvm-rs` — **kayfabe**, a clean-slate Rust rewrite, 142 commits, 57 187 lines of implementation and 53 767 lines of test code, developed 2026-07-24 to 2026-07-29; **it has still never touched a GPU**. This document is pinned to Rust HEAD 15651b1 and C HEAD 4f52877.

What moved since the previous pin (b612a91): the double stopped being polite, and that cost twelve tests. The isolate's defining hazard was, until now, *unmockable*: RM serialises all ioctls per client and waits uninterruptibly, so one wedged verb puts every isolate sharing that client into D-state — while a mock returns promptly by construction. 15651b1 models that as *semantics* rather than sleeps (a per-client lock held for the whole verb, an uninterruptible hold, a positive progress edge). **Forced on unconditionally it makes twelve tests stop terminating and zero assertions fail.** Every one is a liveness claim of the shape "*N pool workers of one isolate have N verbs in flight at once*" — but `l1_os_shell.md` §6.6 states I-NOAMP as a *cross-process* obligation, and `DEFAULT_POOL_WORKERS'` own documentation already says the pool buys latency isolation, not wire concurrency. **The suite's intra-process claim was stronger than the design's, and it passed only because the double returned promptly.** Nothing was narrowed: the modelling is opt-in, the measurement is recorded, and the renegotiation is an owner decision. Parallelism must come from *multiple clients*, not multiple workers on one client.

Second: the C artifact was promoted from history to a standing oracle — and found to be perishable. Two virtualizers facing the same guest driver see identical MMIO iff they behave identically, so a divergence at guest access *N* localises the fault to our reply at *N* − 1: one message, not one layer. Most of the mechanism already existed. What did not exist was a *running* C: recording needs a built QEMU, a pinned 6.8.0-117 guest and host driver 580.159.04, and that deployment had died with its rented box. It was rebuilt from scratch on a fresh GA106 host during this revision, which surfaced a silent build defect (§8.2): `build_qemu.sh` never built or embedded the isolate stub, so QEMU compiled cleanly and came up with *forwarding off*, reporting only `open ct1/gpu FAILED r1=-2 r2=-2` — an error that reads as a missing device node and is not one. Fixed at C 4f52877; the rebuilt bench now reaches `MEMTEST ... PASS` and `GET_SURFACE_PHYS_ATTR rc=0`, i.e. real host GPU memory allocated, mapped and verified through the unprivileged isolate.

Three things stop that from being a victory lap, and they are the reason to read §8.2. (a) The bridge does not deliver compute: `MAP_MEMORY_DMA` is not on the wire at all on this part, so the address table's populate sources live in other crates that are not built. (b) The one control the design expected to carry a page-directory root is not the one an ordinary address space uses. (c) `kayfabe-rmipc` has itself inherited the property it was built to remove — the only manifest naming it is `tests/`. The island moved up one crate; it did not sink.

Also folded in: a live-memory-free defect found and fixed in the core, whose independent oracle carried the identical bug (§8.3); three defects found *in the quality instruments themselves* (§8.4), which is this revision's real theme; and the first mutation campaign over the whole implementation. A pinned document that has been edited past its pin is worse than an unpinned one; every number below was re-measured at 15651b1 (749 tests, all fifteen CI gates clean).

Third, and it bounds the oracle: four measured limits of a C recording. (i) The completion plane has *no* C oracle at all — the C never observes a host completion source, it *forges* completions, and a grep over its isolate verbs finds seventeen call sites and zero poll/event verbs. The plane the current milestone is being built around is therefore invisible to the differential, which is the likeliest source of false confidence. (ii) The diff will never be green end-to-end, and a green diff would itself be the bug: the C has no refusal vocabulary, so every must-differ row is a position where it emits a positive event and we emit a refusal. (iii) Any forwarding-mode trace is non-hermetic by construction — the host GPU DMAs into guest RAM behind every recorder. (iv) The archived logs are unusable: the C's sequence counter is incremented *inside* its log statement, so it counts logged BAR0 accesses only and everything else interleaves by wall-clock. The ordering information was never recorded. ★ The load-bearing good news is that a single-counter total order is nonetheless *sound* — the device creates no thread, bottom half or timer, and all four `MemoryRegionOps` run under the BQL — which upgrades a previously *inferred* assumption to an evidenced one.

Stance. Every number here was measured from the tree while writing, not copied from a document and not inherited from an adjacent sentence — every error previously found in this paper came from restating its own neighbouring prose. Where a project document and the code disagree, the code wins and the disagreement is recorded; there is a list in §10, and it is one of the more useful parts of this paper. Roughly a third of the document is about what does not work, is not built, or is not known.

Contents

1	What this document is, and how to attack it	3
2	The idea	3
2.1	The problem	3
2.2	Vocabulary	3
2.3	Mode 1 and Mode 2	4
2.4	The correctness criterion, and why it is not "replay everything"	4
2.5	Why this is hard	4
3	Provenance: the C artifact, and what it actually proved	5
4	The architecture	6
4.1	Layers and ports	6
4.2	The crate dependency graph	8
4.3	The data path	10
4.4	The isolation model	12
4.5	The lifecycle	14
5	The crates	16
5.1	kayfabe-core — 8 680 lines, BUILT and mutation-hardened	16
5.2	kayfabe-abi — 8 824 lines, BUILT — and mislabelled a stub everywhere	16
5.3	kayfabe-linux-raw — 6 097 lines, BUILT — the entire unsafe surface	17
5.4	kayfabe-gsp — 3 828 lines, BUILT (S0–S5) — and one production crate now calls it	17
5.5	kayfabe-rmrpc — 1 033 lines, BUILT (B0–B4) — the bridge, and the new island	17
5.6	kayfabe-rt — 2 566 lines, BUILT (L1–M1)	18
5.7	kayfabe-fwd — 2 491 lines, BUILT — and never exercised against a GPU	18
5.8	kayfabe-mocks — 2 281 lines, BUILT , test-only	18
5.9	kayfabe-vmm-kvm — 1 891 lines, BUILT , one open defect	18
5.10	kayfabe-isolate — 1 832 lines, BUILT traits; no real implementation	18
5.11	kayfabe-trace — 1 474 lines, BUILT vocabulary; not threaded	18
5.12	kayfabe-shell — 995 lines, BUILT (L1–M2, in progress)	18
5.13	kayfabe-vmm — 927 lines, BUILT traits only	18
5.14	kayfabe-completion — 756 lines, BUILT	19
5.15	kayfabe-util — 675 lines, BUILT	19
5.16	kayfabe-arch — 757 lines, BUILT traits only	19
5.17	kayfabe-mmu — 314 lines, table BUILT ; walker is 41 lines of nothing	19
6	The invariant catalogue	19
6.1	The two-layer trust model	21
7	What is tested, how, and what that does and does not prove	22
7.1	The shape of the suite	22
7.2	The six CI gates	22
7.3	Exercised versus merely present — the project's own core doctrine	23
7.4	The mean test, and composed versus isolated runs	23
7.5	The conservation ledger	24
7.6	The mutation gate — four lies deep, and still no threshold	24
7.7	The OS-free configuration, and why it exists	25
7.8	What the suite does not prove	25

8	Discussion: the good, the bad, and the risky	25
8.1	What is genuinely good	25
8.2	★ The bridge exists — and it does <i>not</i> unlock compute	26
8.3	★★ A live-memory-free defect — and the independent oracle had the identical bug	27
8.4	★★ The quality instruments were wrong — three of them, in one day	28
8.5	kayfabe-gsp: boot is modelled, reboot is not, and one open item has no proposed resolution	29
8.6	Nothing has ever touched a real GPU	31
8.7	The quadratic control plane — an open, measured DoS	31
8.8	Reclamation is the least finished plane	32
8.9	arm64 is measured for the core and unmeasured for the shell	32
8.10	The QEMU ≥ 10.2 floor buys less than the decision that took it assumed	32
8.11	The bench is a single serialised GPU	33
8.12	The recurring failure mode: claims that were true <i>once</i>	33
8.13	Risks this document adds	34
9	How to read the code	35
9.1	Entry points	35
9.2	Conventions	35
9.3	Which design documents to read, in order	35
10	Claims corrected while writing this paper	36
10.1	What this paper could not verify	38
A	Measurement method	39

1 What this document is, and how to attack it

This is a review document. Its job is to describe kayfabe accurately enough that a competent systems engineer who has never seen it can (a) explain what it does and why it is hard, (b) find any component in the tree, (c) judge whether the claims are believable, and (d) find the weak points without help. It is not a status report and it is not a pitch.

Four consequences shape everything below.

Numbers trace to something. Every count, score and measurement names its source. Where a number is an estimate it says so; where a claim could not be verified it is marked **[unverified]** rather than softened. Appendix A gives the exact commands used.

Unbuilt is stated as unbuilt. Most of the planned system does not exist. The build order is deliberate, but a document that blurs “designed” and “running” is worthless for the stated purpose. The layer diagram (Figure 1) and the data-path diagram (Figure 3) both carry explicit **NOT BUILT** bands.

The uncomfortable section is the longest. §8 is where to start if you only read one section.

This document applies the project’s own doctrine to itself. kayfabe’s testing doctrine opens with the rule “a green instrument on an unexercised path is worse than no instrument — it reads as evidence, and nobody re-checks a zero” (docs/design/testing_doctrine.md §1). The same rule applies to a whitepaper. So where this document says a thing is tested, it also says what that test would have to be wrong about for the claim to fail.

The single most important sentence in this paper. No part of the Rust codebase has ever driven a real GPU. There is no binary in the workspace: `find -name main.rs` returns exactly one hit, and it is the offline ABI generator, which lives outside the workspace on purpose. The only composition of vCPU threads, reactor, executor and core that exists is in the test suite (tests/tests/l1_mean.rs, guest_mean_run). **This is still true at this revision,** and the bridge built since the last one does not weaken it by one word. kayfabe-gsp decodes real msgq elements and real RPC envelopes and kayfabe-rmrpc now turns them into RmEvents that a real Gpu applies — but every byte either has ever seen was written by a test into a BTreeMap behind the GuestRam port. **No guest driver has ever posted to any of it.** Everything below describes a system that is real, tested and internally coherent, and that has not yet met the thing it exists to talk to.

2 The idea

2.1 The problem

Giving a virtual machine a real GPU normally means one of two things. **IOMMU passthrough** hands the whole physical device to exactly one guest: simple, fast, and it gives every other guest nothing. **Vendor virtualization** (SR-IOV, NVIDIA vGPU) shares the device properly, but it exists on datacenter parts under licences, not on the consumer cards most people own.

kayfabe is a third option: **share one commodity GPU among several untrusted virtual machines, with per-guest-process blast-radius containment, using only unprivileged host processes.** The multi-tenant part is the point. A single cross-tenant leak, or a hostile guest that can wedge the box, would be fatal to the thesis.

2.2 Vocabulary

The rest of this paper leans on five terms.

- **RM** — NVIDIA’s *Resource Manager*, the object model inside the driver. Everything a GPU program touches is an RM object with a handle: clients, devices, address spaces, channels, memory, engine contexts. Guest software allocates and frees them through ioctls and RPCs.
- **GSP** — the *GPU System Processor*: on modern NVIDIA hardware a real CPU core on the GPU die, running a firmware copy of much of the resource manager. The host driver no longer programs most of the hardware directly; it sends RPC messages to the GSP over a ring buffer in memory.
- **VAS / PDB** — a GPU virtual address space, and its *page directory base*: the physical address of the root of that address space’s page tables. The GPU’s MMU keys page tables by PDB, which makes **the PDB the real memory boundary** — not the client handle.
- **Channel / vChid** — channels are the GPU’s work-submission queues; the *virtual channel id* is how a submission is routed to one. **vChid is the execution boundary.**
- **Doorbell** — the MMIO register write that says “channel *N* has work waiting”. The value written is a token that encodes the vChid.

2.3 Mode 1 and Mode 2

The predecessor project defines two modes, and kayfaber is only the second.

Mode 1 forwards the guest's *userspace* NVIDIA ioctls to a host stub that replays them against the real `/dev/nvidia*`. It needs a guest kernel module and a virtio transport, so the guest is modified, and it caps you at "a Linux guest running our module". It is mature: 22 real GPU applications at host parity, Vulkan, OpenGL, a working desktop present path, five security audits.

Mode 2 inverts it. The guest runs the **stock, unmodified NVIDIA kernel driver** against an *emulated* PCI device that presents a plausible GPU and a *faked* GSP. We implement the RPC ring the driver talks to and answer as the firmware would. The guest driver believes it owns hardware: it allocates RM objects, builds GPU page tables, creates channels, and rings doorbells. We are on the other side of that conversation, and from the protocol traffic we reconstruct what the guest program was actually trying to do — then do the equivalent thing on a real host GPU through ordinary unprivileged userspace operations, inside a sandbox. Mode 2 needs *nothing* from the guest, which is the entire reason it is worth the difficulty.

2.4 The correctness criterion, and why it is not "replay everything"

The naive reading of Mode 2 is "intercept the guest's privileged operations and perform them on the host". That is wrong, and the project's forwarding model says so explicitly:

*"The job of Mode-2 is **not** to replay those low-level steps on the host. It is to **recover the original userspace-level intent and re-express it as the normal, unprivileged host userspace operations** — exactly the operations Mode-1 forwards directly."*
[C: docs/design/mode2_forwarding_model.md]

The guest's kernel RM has *already* decomposed an unprivileged userspace intent (`cuCtxCreate`, `cuMemAlloc`, a kernel launch) into privileged GSP-internal steps. Replaying those steps on the host would require privilege we deliberately do not have — and the project has a named lesson about this: an `NV_ERR_INSUFFICIENT_PERMISSIONS` from the host stub *never* means "we lack a privilege we should have", it means "we forwarded at the wrong layer". So the operational split is:

- **Case 1** — the RPC essentially *is* the userspace operation. Re-issue it roughly 1:1 on the host, through the isolate, unprivileged, and let the host's own kernel RM legitimately re-derive the privileged steps for itself.
- **Case 2** — a GSP-internal control with no userspace equivalent (`PROMOTE_CTX`, `GET_CTX_BUFFER_INFO`). **Acknowledge the guest; do nothing on the host.** The host has already done the equivalent for its own context.

The correctness criterion that licenses this is **observable end-state equivalence**, not step fidelity. It is perfectly correct for some guest-kernel operations to complete instantly as fakes, *as long as* the one operation that actually carries the work triggers the real host-side chain. The hard boundary on the other side is equally explicit: a completion that gates guest *userspace* must be a real host write. Forging one would make the whole project a lie, and the design forbids it by name.

2.5 Why this is hard

1. **The protocol is not documented and not stable.** It varies on two independent axes: the *driver version* (wire layouts) and the *GPU generation* (silicon behaviour). kayfaber separates these as "Axis A" and "Axis B" and gives each its own trait seam, because conflating them is its own bug class.
2. **Some things cannot be faked at all.** GR's golden context is produced by FECS/GPCCS microcode on real silicon. There is no way to fabricate one. The design's response is to never emulate an engine — forward the guest's own pushbuffer and let real silicon produce the context.
3. **Address translation is a trust problem, not a lookup problem.** The guest builds GPU page tables in memory it controls. Reading them at an arbitrary instant means acting on bytes an adversary can change under you.
4. **Identity collides by construction.** Guest handles and guest VAs are per-process namespaces. Two processes using identical values is normal. Anything that keys on them is wrong (Figure 4).
5. **The failure modes are global.** A mis-resolved address is not a wrong answer; it is a host GPU MMU fault that takes out a channel, and in the worst case a state the guest driver cannot recover from without a full VM restart.
6. **Everything is concurrent.** *N* guest vCPUs, several guest processes, several host isolate processes, an interrupt path, and a host RM that *serializes every ioctl per client* so that one wedged verb blocks every

isolate in D-state.

3 Provenance: the C artifact, and what it actually proved

kayfabe is a rewrite, not a greenfield project. The C artifact at `/workspace/nvidia-gpu-passthrough` is the working prior implementation and the differential oracle for everything the rewrite claims about NVIDIA's behaviour. Being precise about what it did and did not prove is the foundation of every other claim here.

Claim	Mode	What was actually measured
22 real GPU apps at host parity	1	True. <code>tests/perf/realapp_matrix.md</code> , 2026-06-01. Same binary, host and guest, strictly serially on one RTX 3060, with a correctness check alongside throughput; parity gate ≥ 0.90 . 10 CUDA benchmarks, 7 PyTorch, 2 LLM, 3 graphics. Caveat the doc states itself: NVENC is <i>partial</i> , not parity (<code>InitializeEncoder</code> fails); NVDEC excluded (broken on the host baseline too).
7B LLM at 63 tok/s	1	True but mis-framed. 63.4 tok/s is a Mode-1 A/B endpoint after a caching fix, not a headline result. The audited same-day parity figure is host 67.8 $\rightarrow$ guest 66.0 tok/s , $0.97\times$ (Qwen2.5-7B Q4_K_M, -ngl 99, RTX 3060). Quote the ratio.
Mode-2 bare-metal parity	2	True, one workload. 49.9 tok/s Mode-2 vs 47.5 tok/s host-native <i>on the same RTX 3050</i> , i.e. zero forwarding overhead within noise. The entire earlier 20 $\rightarrow$ 50 tok/s gap was nested-virt vmexit tax , not Mode-2 design — the <i>vast.ai</i> bench is itself a VM, so the Mode-2 guest was L2. The project retracted its own earlier “~21% residual” claim as an apples-to-oranges comparison across two different GPUs.
Mode-2 CUDA correctness	2	True. Full fake-boot + GSP RPC + GMMU + CE + IRQ; <code>cuInit</code> , <code>cuCtxCreate</code> , byte-exact <code>matmul</code> ; PyTorch 2.5.1 <i>single-process</i> . Bugs #12 (second-context hang) and #13 (multi-iteration hang) resolved on hardware.
What the C artifact never proved		
Mode-2 multi-process	2	Never. Bug #14 is open. Root-caused on hardware 2026-07-24 by a host Xid 31 <code>FAULT_PDE</code> naming the loser's exact GR channel: the second process's identical guest VAs were never published into <i>its own</i> host address space.
Mode-2 sequential processes	2	Broken at HEAD. Measured 2026-07-25: exactly <i>one</i> CUDA process works per QEMU lifetime; the second fails <code>cuInit</code> $\rightarrow$ 999 regardless of how the first exited. The “fresh boot per clean run” bench discipline <i>is</i> this bug, not a precaution.
Mode-2 graphics / display	2	Never. All Vulkan/OpenGL/present results are Mode 1.
Mode-2 on the 22-app matrix	2	Never run. It is Mode-2's definition of done.
Multi-GPU, MIG, non-Ampere	—	Never. Ampere GA106 only. Turing/Ada/Hopper/Blackwell rows in the ABI doc are all marked <i>assumption</i> — <i>verify</i> .
Mode-2 security posture	2	Not audited. The five security audits are Mode-1-era. The 2026-07-25 bench found live guest-reachable Mode-2 defects, including parsing arbitrary guest RAM as GSP RPC elements after a guest-triggered driver reload.

The structural reason for the rewrite. #14 is not a bug that could be patched. The architecture document states it in one line: “*the emulator's completion, execution, isolate, and GPA-arena planes are each a single-process singleton, and they fail in dependency order.*” Four rounds of investigation each found a different plane; they were one structural fact observed four times. Making per-process separation structural rather than retrofitted is the rewrite's founding requirement, and it is why the diagram in Figure 4 is the centre of the design.

One provenance correction worth carrying. Two C design documents state the C baseline encodes “~14 months of quirk capture”. Git says the first commit is 2026-05-24 and the last is 2026-07-24: **two months, 739 commits**. Either the figure means agent-hours, or it is wrong. Use the verifiable number.

4 The architecture

4.1 Layers and ports

The one defining constraint. The logic core is a pure state machine over guest-supplied bytes: no OS, no syscalls, no hypervisor types, no wall clock, no `#[repr(C)]` NVIDIA struct layouts, and no concrete GPU-generation or driver-version name anywhere. Everything effectful crosses a trait seam. `unsafe_code = "forbid"` is a workspace lint; every core type is compile-time-asserted `Send + Sync`.

This is enforced, not intended. Four CI greps run on every push: a *hexagonal boundary* grep that fails if `eventfd|epoll|timerfd|rawfd|libc|O_NONBLOCK` appears in any of the 11 pure crates; a *VMM-vocabulary* grep that fails if any QEMU concept (`BQL`, `MemoryRegion`, `qemu_bh`, `iothread`, ...) appears in those 11 plus `kayfabe-rt`; a *generation-name* grep over 10 of them that fails on any concrete chip or GPU-generation name outside a comment; and a *GPA-accessor* gate that asserts `.gpa_read(/.gpa_write(` appears **exactly once** in the entire forwarding crate, and **nowhere** in the other nine.

The claim that pays for this constraint is the **adapter contract**: bringing up a new GPU generation is `impl Arch` for `<Gen>` in an adapter crate with *zero edits to any logic crate*. The standing proof is that the whole suite runs the real core against `MockArch`, whose encodings are deliberately non-NVIDIA — if the core had absorbed a real encoding anywhere, the mock would not work.

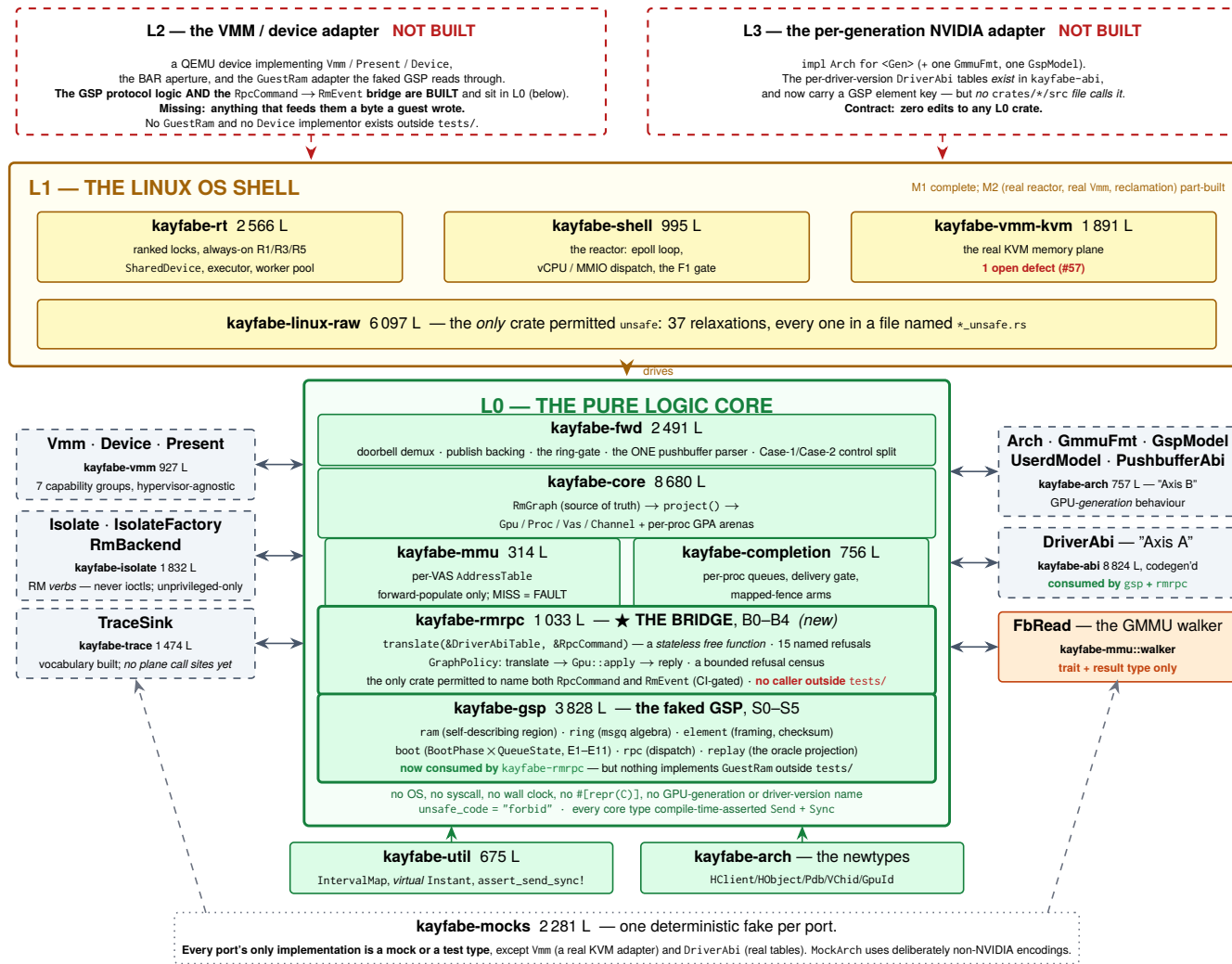


Figure 1: The layer model and the hexagonal ports. Line counts are measured at b612a91 (crates/*/src/**/*.*.rs; kayfab-abi's figure includes its detached offline generator). Implementor census, counted from impl <Trait> for across the tree: FbRead has no implementation of any kind; Device, Present, Arch, GspModel, Isolate, IsolateFactory and RmBackend have only mocks or test types; Vmm has one real adapter (KvmVmm); TraceSink has three in-crate implementations; CommandPolicy has two, one of which (GraphPolicy) is the new bridge and is production code. kayfab-rmrpc joined the L0 band at this revision, between gsp and core — but the L2 shell that would feed either of them still does not exist.

4.2 The crate dependency graph

Figure 2 is derived from every `Cargo.toml` in the tree and cross-checked against `Cargo.lock`; it is not drawn from the design documents. Three findings come straight off it.

The external dependency surface is essentially zero. In the whole workspace there are three external crates: `libc` — a real dependency of `kayfabe-linux-raw` only, and a *dev*-dependency of `kayfabe-vmm-kvm` and tests for asserting exact errors — plus `trybuild` and `proptest`, both dev-only. **Fourteen of the seventeen** crates have no external dependency of any kind (the new crate added none). Whatever else is true, the supply-chain surface is not a place to look for problems.

The graph is acyclic, including dev edges, which the strictly lower-triangular shape of the matrix demonstrates directly. *The ordering that makes it lower-triangular had to change again at this revision*, for the second revision running: `kayfabe-rmrpc` is new and sits below `core` and `gsp` because it depends on both, and `mocks` acquired an abi edge (2dac655 promoted `WireClassArch` into it rather than transcribing `NV01_ROOT` a fourth time). The re-ordering is the matrix's own record of the shape changing.

But the claimed strata are not disjoint, and the documents imply they are. `kayfabe-mmu` — a core crate — depends on the `kayfabe-isolate` *port* (it needs `HostHandle`). `kayfabe-core` and `kayfabe-fwd` depend on the *trace* and *vmm* ports. `kayfabe-gsp` — also a core crate — depends on the *abi* and *trace* ports. “Core” and “port” are one DAG with a purity rule, not two layers. That is a defensible design (the trace crate had to sit *below* core so every plane could emit into it), but a reader who takes the hexagon picture literally will be surprised by the manifests.

★ **This paper has now been wrong about the island question twice, in opposite directions, and the third answer is the interesting one.** The first revision called `kayfabe-abi` and `kayfabe-gsp` “disconnected islands”. The previous revision corrected half of that and sharpened the other half into this document’s headline: *“kayfabe-gsp is still an island in the direction that matters — its output type `RpcCommand` has zero references anywhere in the tree outside the crate that defines it, so the critical path is the bridge, not the crate.”* **That sentence is now false.**

Re-measured against the manifests and the source, not inherited. `RpcCommand` is named in four files outside `kayfabe-gsp`: `crates/kayfabe-rmrpc/src/lib.rs`, `.../policy.rs`, and `tests/tests/rmrpc_bridge.rs`. `kayfabe-rmrpc` is a production crate, it depends on both `kayfabe-gsp` and `kayfabe-core`, and it is the *only* crate permitted to name both `RpcCommand` and `RmEvent` — which is not a convention but CI step 12, a *bridge-exclusivity* grep added in the same commit. So `kayfabe-gsp` has a production consumer, and the chain `bytes → FSM → RpcCommand → RmEvent → Gpu::apply` exists entirely inside production crates.

And the property moved up exactly one crate rather than disappearing. The only manifest that lists `kayfabe-rmrpc` is `tests/Cargo.toml`. Nothing constructs a `GraphPolicy` outside `tests/`; nothing implements `GuestRam` or `Device` outside `tests/`. 1 033 lines of bridge now sit on top of 3 828 lines of protocol logic, and the composite still has no caller that is not a test. That is a materially better position than the last revision described — the seam is built, typed and gated, and its shape has been argued against the compiler two dozen times (§8.2) — but a reader looking for the critical path should now be looking one layer higher again, at L2.

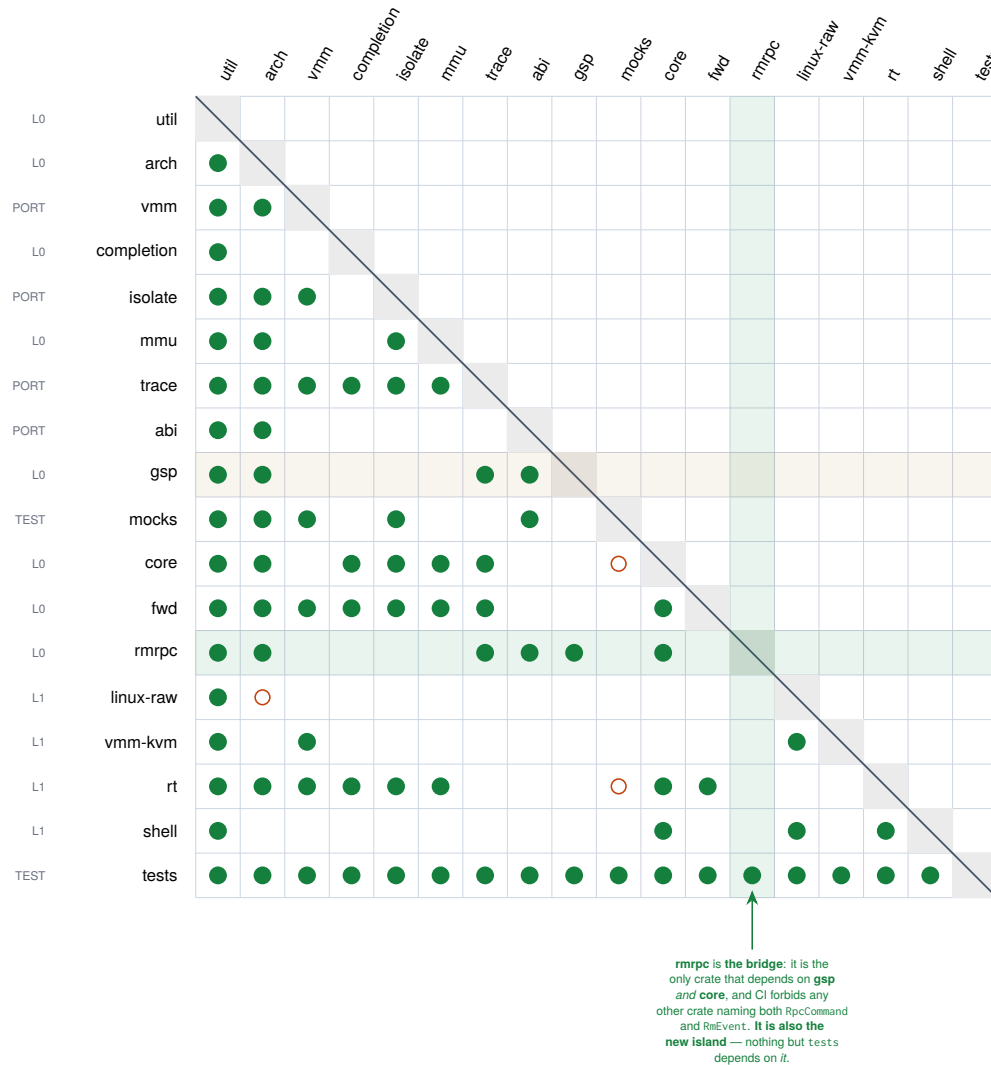


Figure 2: The crate dependency matrix, re-derived from scratch from the manifests at b612a91 — 18 members, 80 [dependencies] edges and 3 [dev-dependencies] edges. kayfaber-rmrpc is the new row, and it is the whole story of this revision: it depends on gsp *and* core, which is what makes it the bridge, and nothing but tests depends on it, which is what makes it the new island. The previous revision's matrix is superseded rather than corrected — it was accurate for the 17 crates it drew.

4.3 The data path

Figure 3 traces one guest GPU operation from a CUDA call to real silicon and back, naming the symbol each step lives at. Three things about it are worth stating in prose.

The plan/execute/commit shape is the whole concurrency design. A host RM verb can block for a long time — the host RM serializes every ioctl per client, and it waits *uninterruptibly*, so one wedged verb parks every isolate that shares the client in D-state. Invariant **R1** therefore forbids issuing a verb while any ranked lock is held. That forces a three-phase shape: *plan* under locks, *execute* with no lock held, *commit* after re-acquiring. And because the locks were dropped in the middle, invariant **R5** requires the commit to re-validate everything the plan assumed. This is not free — §8 covers what it costs.

The ring gate is the #14 fix, and it is now structural in the type system. Before any host operation, a doorbell submission’s entire working set must already be published into *this* *Vas*’s own host address space. If it is not, the call refuses with `NoVas` having issued zero host operations. The failure it prevents is exactly the one the C artifact hit on real hardware: the losing process’s identical guest VAs were never published into its own host GR address space, so the host GPU faulted past the shared prefix. At the previous pin this was a call-graph property — “every path happens to go through `plan_doorbell`”. At 634b253 it stopped being one: `VerbPlan::Doorbell` is `#[non_exhaustive]`, so no struct expression for it exists outside `kayfabe-isolate`, and the sole constructor `VerbPlan::gated_doorbell` runs the gate. Constructing an ungated doorbell is now compile error E0639, pinned by a `trybuild` row (`crates/kayfabe-isolate/tests/ui/`).

The upper half of the diagram gained its missing link, and what remains missing is now a different kind of thing. Two revisions ago there was no register model, no GSP message-queue transport and no RPC decode; one revision ago all three existed but nothing consumed their output. Both gaps are closed: `kayfabe-rmrpc::translate` turns an `RpcCommand` into an `RmEvent` and `GraphPolicy` applies it, from inside the FSM’s own `service_command_queue`. The sentence that survived the last two revisions verbatim — “*nothing in the repository produces an `RmEvent` from real guest bytes*” — is **retired**. Its correct replacement is narrower and harder to fix:

Something now produces an `RmEvent` from wire bytes. Nothing produces the bytes. The chain is real and lives in production crates, but its input port `GuestRam` and the `Device` that would own the BAR have no implementor outside `tests/`, so every byte that has ever traversed it was authored by the suite. And *even fully fed*, this chain does not reach compute: the control plane’s address-table populate sources are not on it (§8.2).

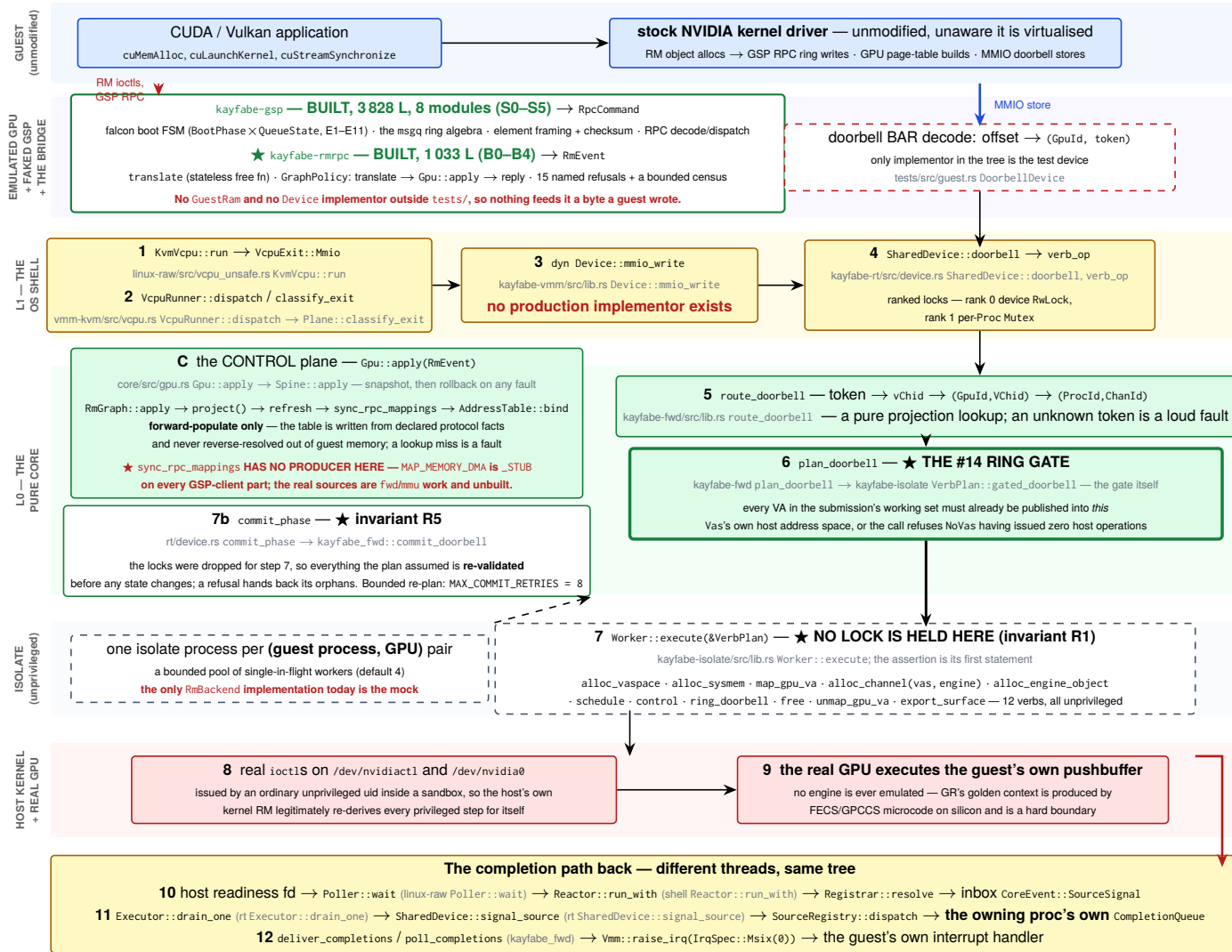


Figure 3: The data path, verified function by function against the tree at b612a91. Numbered steps are the execution plane; **C** is the control plane, which populates the address table from declared protocol facts. Boxes name *symbols*, not line numbers: the repository converted its own 105 pins to symbols in 3fd1455 after they drifted. The emulated-device band changed most at this revision — the bridge box is new, and so is the red note recording that the address table's two real populate sources are *not* reachable through it. Red text marks what does not exist.

4.4 The isolation model

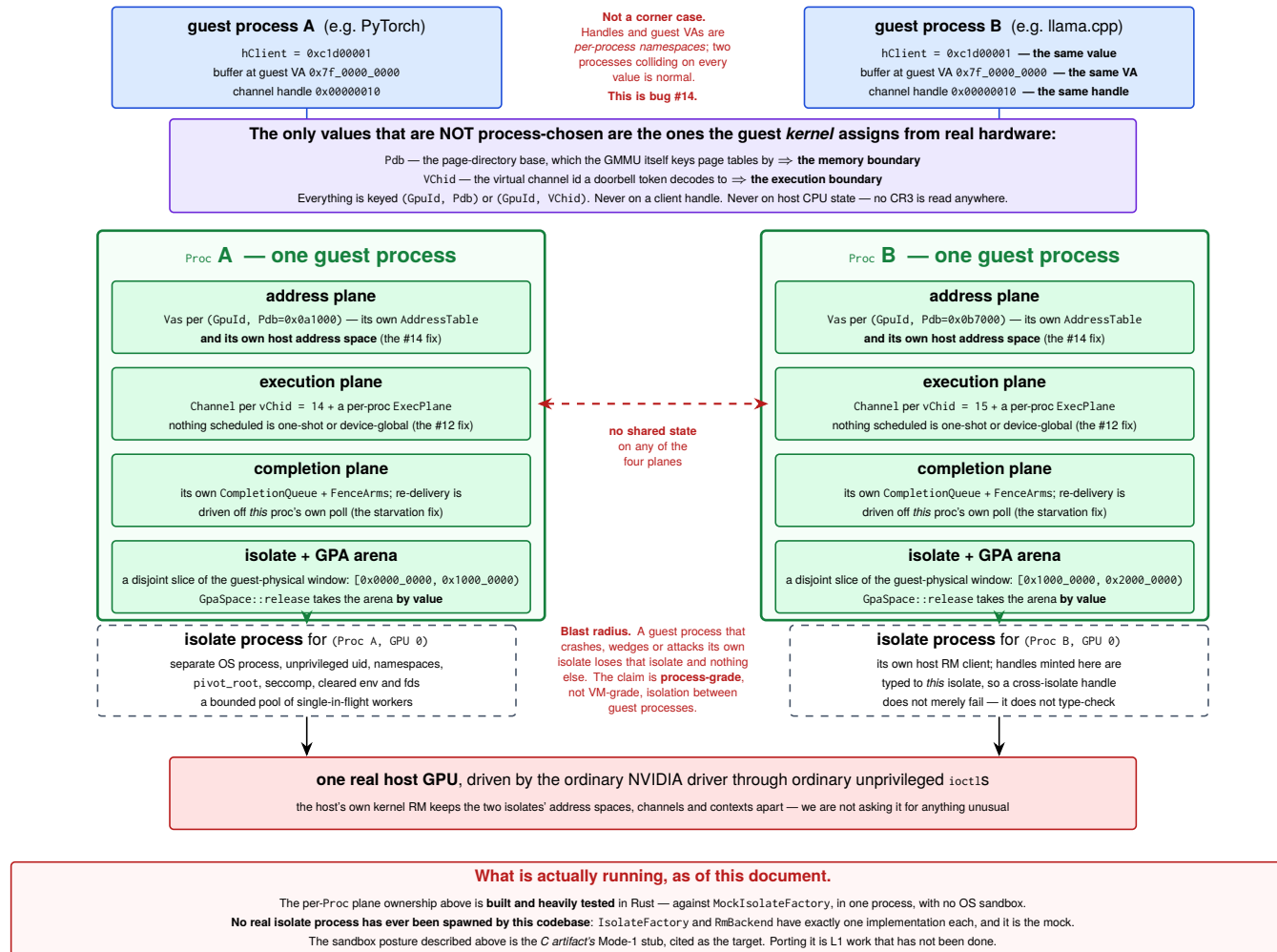


Figure 4: The process and isolation model, and the problem it exists to solve. The per-Proc ownership of all four planes is built and heavily tested against mocks; the sandboxed isolate process at the bottom has never been spawned by this codebase.

The rule that generates the whole model is one sentence: **a guest process is a grouping label, and it is never what an address or execution operation keys on.** Address operations key on (GpuId, Pdb); execution operations key on (GpuId, VChid). Proc exists only to own the isolate, the GPA arena and the lifecycle.

Proc membership is itself a **pure projection** of the RM graph — the dup-connected components of declared user clients — never inferred from timing, never from arrival order. That matters because it means two identical event streams delivered in different orders produce bit-identical process boundaries, which is a property the test suite checks directly by permuting streams.

4.5 The lifecycle

The law every path obeys: RETIRE EAGER, REAP DEFERRED		
Retire is immediate and cheap: the Proc stops accepting operations, its isolates refuse new check-outs. Reap is the heavy half — freeing host objects, killing isolate processes, recycling the GPA arena — and it runs later, at a quiesce point the adapter declares (Gpu::reap_retired). Why: the C artifact proved that reaping at the client-root free hangs the dying context's own residual polls. Reaping never is the #80 leak. Both were paid for on hardware.		
TRIGGER	WHAT HAPPENS	STATUS — read from the tree
T0 a guest frees a subset and keeps running	The most frequent path in a real workload, and the one with no backstop — nothing dies, so nothing reclaims. Dropped host identities move to a pending_release queue on the Proc and drain on a checked-out worker.	Built, and lazier than designed. The drain may fire only when the isolate is otherwise idle (is_quiesced), because no lock can exclude a verb that runs lock-free.
T1 a verb is cancelled mid-flight	The cancel is latched, discharged with no lock held, surfaces as EINTR → Interrupted → FwdFault::Cancelled; the chain's own unwind frees what it had allocated; the worker is checked back in.	Built and tested (tests/tests/cancellation.rs).
T2 a guest process exits — normally, killed, or killed mid-verb	All three are one path: the guest kernel frees the client root on fd close, so an RmEvent arrives, the projection finds no boundary, and the retire is graph-driven . Every checked-out worker is cancelled; fresh handles become Orphans; the Proc lands on the retired list awaiting T3.	Built and tested.
T3 the GPU goes idle — THE quiesce point	The guest <i>tells</i> us: its driver emits UNLOADING_GUEST_DRIVER (RPC fn 47) on an idle release and re-runs the queue handshake. That re-handshake is the quiesce point, and the C already ran its deferred reap there. Same doctrine as the address table: an observed protocol event, never an inference about idleness.	HALF A LAW. reap_retired exists and CoreEvent::Deferred(DeferredReap) exists — and nothing arms either . Reap-deferred with no trigger is indistinguishable from reap-never in a run that never quiesces.
T4 the guest driver restarts / the GSP re-handshakes	On rmmmod, guest panic or VM reset the guest sends <i>no</i> Free events, so the graph, every live Proc, its isolates, arenas, host handles and routing maps all survive into the next driver life — which then derives a second set of components beside the corpses.	NOT BUILT, and the obvious trigger does not exist. Bench-measured 2026-07-26: rmmmod emits no fn-47 at all — the idle release at process exit already consumed it. The unload has no observable signal.
T5 an isolate worker dies	Routing is settled (the component is condemned, keyed on its client set). Reclamation is not: a worker that died <i>mid-chain</i> left allocations that are in no Orphans and in no core state. The answer is to escalate to isolate death — SIGKILL — because killing the process is how you reclaim state you cannot enumerate.	Partly built. The design calls this "the single most leak-prone moment"; the carrier type (VerbFailure{err, orphans}) exists.
T6 isolate garbage collection	SIGKILL → the process's <i>exit descriptor</i> becomes readable → reactor → waitpid (non-blocking) → close descriptors, tear down every double-mmap, release arenas, drop worker slots. The kernel already released the host RM objects by closing the connection.	Designed; the OS half is unexercised — no real isolate process has ever been spawned.
T7 whole-device teardown	An explicit ordered shutdown(), never a Drop: stop accepting traps, cancel every worker, wait bounded, retire everything, reap unconditionally, kill every isolate, drain the inbox asserting every remaining event faults rather than mutates, join the threads with no lock held, and assert the conservation ledger balances . Drop is a tripwire that logs loudly if shutdown() was skipped.	Designed; partly built.
The instrument all eight are measured by — the conservation ledger HostLedger, kayfabe-mocks/src/lib.rs:1241 Replays the recorded verb log and demands: every acquisition matched by exactly one release, nothing released that was never acquired — objects <i>and</i> mappings. A test may declare a ResidueClaim for what it knowingly leaves outstanding, and the counts are compared for equality, not as a bound : less residue than declared fails too, so a fix that closes a leak must delete the claim that excused it. ★ Its first census over the mean run reported zero leaks — and that was a true negative, not a pass. The workload it was measuring only ever added and removed <i>RPC</i> bindings, which own nothing host-side. Once a phase was written that actually allocates before it frees, the honest baseline was 24 objects, 6 mappings and 24 576 GPA bytes leaking, linear in frees .		

Figure 5: The eight teardown triggers and the retire/reap split. Two of the eight are honestly broken and say so: T3’s quiesce trigger is designed but nothing arms it, and T4’s obvious trigger was measured on hardware and does not fire.

The lifecycle is where the C artifact’s scar tissue is thickest, and it is worth understanding why the split exists. Reaping resources at the moment the guest frees its client root *hangs the dying context’s own residual polls* — bench-proven. Not reaping at all exhausts the shared guest-physical window after a handful of process generations — also bench-proven, as bug #80, and *re-introduced and re-found in the Rust core* by the regression test written for it. Hence: retire eagerly (refuse new operations, cheap), reap later at a quiesce point the adapter declares.

Figure 5 marks two of the eight triggers red, and the reasons are different in an instructive way. T3 is a design that is not wired: `reap_retired` exists, `CoreEvent::Deferred(DeferredReap)` exists, and nothing arms either. T4 is worse — the trigger the design chose *was measured on hardware and does not fire*. The design assumed `rmmod` emits RPC function 47; the bench showed it emits none, because the idle release at process exit already consumed it. The problem is not that two triggers are indistinguishable; it is that the driver unload has *no* observable signal.

5 The crates

Sizes are `crates/<name>/src/**/*.rs`, measured at b612a91. “Maturity” is this document’s reading of the tree, cross-checked against `docs/design/crate_maturity_map.md`.

5.1 kayfabe-core — 8680 lines, **BUILT** and mutation-hardened

Owns the spine. `RmGraph` is the source of truth: a refcounted resource/handle split with `DUP_OBJECT` aliasing, order-tolerant “parked” facts (a mapping that arrives before its target is held, not rejected), mapping refcounts and capacity bounds. `project()` is a pure function from the graph to process boundaries and the two routing tables. `Gpu::apply` is transactional — snapshot, mutate, re-project, roll back on any derivation fault, so a hostile event earns only its own refusal. `gpa.rs` owns the guest-physical window and carves per-process arenas; `GpaSpace::release` takes the arena *by value*, which makes releasing a live process’s arena unrepresentable rather than merely wrong.

Deliberately not here: anything about wire formats, any NVIDIA constant, any OS call.

And it grew by 205 lines at this revision for one reason worth naming: 1019064 fixed a live-memory-free defect in `RmGraph::apply`, described in §8.3. It is the sharpest single defect the project has found in its own core.

5.2 kayfabe-abi — 8824 lines, **BUILT** — and mislabelled a stub everywhere

Axis A: the per-driver-version wire layouts. Contains an offline generator (`crates/kayfabe-abi/gen`, a separate cargo project with zero third-party dependencies) that parses NVIDIA’s own FINN-emitted C headers from the open kernel modules and emits Rust; 13 generated structs plus one hand-transcribed variant; a version-dispatch table keyed on full `major.minor.patch`; and a large oracle suite. The `#[repr(C)]` mirrors are a *layout oracle only* — decode is `from_le_bytes` out of a byte slice, never a cast, so `forbid(unsafe_code)` still holds.

Its test design is the best in the repository: layouts are pinned against **three independent oracles** — NVIDIA’s own headers, `gVisor`’s independent transcription, and the C artifact — with a `file:line` citation per number, and a test whose name is `nvos46_the_three_oracles_disagree_and_here_is_exactly_how`. That test found that the C artifact’s *parity test* was weaker than the C code it was supposed to guard.

It now has two consumers — `kayfabe-gsp` and, since 76a0077, `kayfabe-rmrpc`, which is the crate that keeps every NVIDIA constant quarantined here: the bridge names none, by design and by review. 712e370 added the per-class alloc table (`DriverAbiTable::alloc_params`), six class ids and four params decoders (`TSG`, `CtxShare`, `Channel`, `Device`).

The previous revision’s standing criticism of this crate has half expired, and the surviving half is now the sharper one. That criticism was: “`DriverAbiTable` carries only `version`, `map_dma` and `note`; there is no key for the GSP element layout, so no production path selects one.” **The first clause is false at this revision** — 250b78c added `gsp_element` and `gsp_init_args` keys, with a `GspElementWire` discriminating 580’s 48-byte header from 610’s 16-byte one, and the element-size minimum and maximum moved out of a test fixture into the table (before that, *the fixture was the only definition of them*). **The second clause is still true, and re-checked by grep rather than inherited:** `gsp_element_wire()` and `gsp_init_args_wire()` have no caller in any `crates/*/src` file — only `crates/kayfabe-abi/tests/`. The version-dispatch machinery is a built capability with a test-only interrogator.

5.3 kayfabe-linux-raw — 6097 lines, **BUILT** — the entire unsafe surface

The only crate permitted `unsafe`, and the only one that does not inherit the workspace lints (because `forbid` cannot be relaxed by an inner `allow`). It restates `unsafe_code = "allow"` plus `clippy::undocumented_unsafe_blocks = "deny"`. Every file that uses the keyword is named `*_unsafe.rs`, so `ls` enumerates the audit surface: 7 files, **37 relaxations**, held by a CI ratchet (`EXPECTED_UNSAFE_BLOCKS=37`) that fails on any change in either direction. It also carries a `trybuild` compile-fail matrix — 9 files that must *not* compile — covering things like a bare `MAP_FIXED` address, a page-size literal, and a view outliving its region.

5.4 kayfabe-gsp — 3828 lines, **BUILT** (S0–S5) — and one production crate now calls it

The faked GSP: the guest driver’s entire picture of its GPU arrives through this crate. Eight modules. `ram` owns the guest-RAM port and walks the shared region’s *self-describing page table* (the region is allocated `NV_MEMORY_NONCONTIGUOUS`, so the C artifact’s `base + offset` arithmetic is a latent bug this crate does not reproduce). `ring` is NVIDIA’s `msgq` algebra — geometry, cursors, and the `msgqRxlLink` acceptance predicate reproduced code-for-code so “would the guest link this header?” is answerable offline. `element` is framing, the 64-bit XOR-fold checksum and multi-element split/join. `boot` is `BootPhase × QueueState` with transitions E1–E11, and it deletes all three of the C’s one-shot latches — two by construction, one by making the queue GPA exist only inside a `Bound` variant. `rpc` classifies functions into three dispositions (must-reply-synchronously, reply-expected, must-*not*-reply). `replay` is the differential projection its oracle compares, plus a ledger in which a deliberate divergence is admissible only with four things attached, one of them an independent oracle.

Five things it is deliberately not tied to, each with a test that pushes on it: a GPU architecture (every offset is behind `kayfabe_arch::GspModel`; the suite drives the same FSM through two deliberately-different models), a driver version (the element layout is an `ElementLayout value`, not a constant), a hypervisor (the only way in is an abstract register access and the `GuestRam` port; CI greps for every VMM type name), a single GPU lifetime (`GspFsm::device_reset` rebuilds the value and `teardown` drops the queue binding *by value*, so driver-restart works in-process, repeatedly), and a CPU architecture (every wire access is explicitly little-endian).

Two things changed underneath it at this revision. It acquired a consumer (`kayfabe-rmrpc`, below) — and 250b78c implemented **GSP-S1**, the guest-kernel memory-safety invariant that the previous revision of this paper singled out as “the most security-relevant thing in this document that is a sentence rather than a mechanism”. It is now a mechanism: `max_elements` is *derived* from the layout rather than declared, `encode_message` refuses `ElementCountOutOfRange` before building a run *unconditionally* (not gated on the layout’s field, because the guest’s staging buffer exists at every version), and `service_command_queue` range-checks the guest’s declared count and cross-checks it against the derivation. §8.5 is about what remains.

5.5 kayfabe-rmrpc — 1033 lines, **BUILT** (B0–B4) — the bridge, and the new island

New at this revision, and the reason this paper’s previous headline is retired. Two modules. Its whole entry point is one signature:

```
translate(&DriverAbiTable, &RpcCommand) -> Result<Translation, BridgeRefusal>
```

and it is a **free function** — no `&mut self`, no handle table, no memo, no seen-set, no guest-memory access and no lookup. That statelessness is the design’s central claim rather than a tidiness preference: a per-handle cache here would re-open the whole recycled-handle bug class the core closed the same week, so *recycle-safety is structural rather than tested*. The suite pins it by mutation — a per-handle memo and a seen-set are planted as deliberate mutants at every stage, and by B3 they were visible to 20 and 24 tests respectively.

`policy.rs` holds the only thing in the crate that applies: `GraphPolicy`, the `kayfabe_gsp::CommandPolicy` the boot FSM calls, whose `respond` is `translate → Gpu::apply → reply`. A refusal answers with a non-zero status and an **empty** body — deliberately not the request echoed, because reflecting guest bytes under a failing status is `memcpy(resp, cmd, 4096)`, the C defect this deviates from by name. Refusals are counted by a `RefusalCensus` bounded by the finite tag set rather than by traffic, with `applied()` and `inert()` kept as *two* counters so a regression that turned every `alloc inert` cannot hide inside a sum.

What it deliberately is not: it is not in `kayfabe-fwd`, where both the port plan and `rpc.rs`’s own comment said the bridge would go — `fwd` depends on neither `abi` nor `gsp`, and B1 needed no `Worker`, `Vmm` or `RmBackend` at all. Three superseded placements are recorded in the design doc rather than quietly dropped.

Its honest limits, stated as the crate states them: it has **no caller outside** tests/; it cannot emit a `Translation::Forward` because the control-classification table it would hand to does not exist, so the entire control long tail is refused by name; and it cannot reach the compute path at all (§8.2).

5.6 kayfabe-rt — 2566 lines, **BUILT** (L1-M1)

The threaded shell. `LockRank{Device=0, Proc=1, Leaf=2}` with an always-on (not `debug_assert`) rank check that panics *before* the OS acquire, so an inversion is deterministic. `SharedDevice` is the plan/execute/commit driver, and it ships **both** lock modes — `Sharded` and `Degenerate` — from day one, tested as first-class configurations with a differential asserting a bit-identical end state, specifically so a late granularity change is never the untested mode.

5.7 kayfabe-fwd — 2491 lines, **BUILT** — and never exercised against a GPU

The entry points an adapter calls: `handle_doorbell`, `publish_backing`, `parse_pushbuffer` (the one parser: CE page-table-write capture, semaphore release, TLB invalidate, everything else opaque and passed through), `forward_engine_object` and `route_control` (the Case-1/Case-2 split), the fence arms, and the present route. This is the crate whose correctness is least established, because its correctness is a claim about NVIDIA's behaviour and nothing here has met NVIDIA.

5.8 kayfabe-mocks — 2281 lines, **BUILT**, test-only

One deterministic fake per port, plus the shared verb recorder and the conservation ledger. Excluded from the mutation denominator on the argument that mutating an instrument is a category error — a surviving mock mutant is not a coverage hole. **That exclusion was inert until 2026-07-28** and every figure ever quoted “excluding scaffolding” was a hand subtraction; see §8.4. *Strictly*, the crate is not confined to dev-dependencies: `tests/` and `fuzz/` list it as a normal dependency, so `cargo build --workspace` compiles it. Nothing shipping links it. It gained an abi dependency at this revision to host `WireClassArch` — the one Arch seam whose argument arrives off the GSP wire, so a test arch that cannot classify a real NVIDIA class id cannot exercise the bridge.

5.9 kayfabe-vmm-kvm — 1891 lines, **BUILT**, one open defect

The real KVM memory plane: real `/dev/kvm` file descriptors, real memslots, real vCPU run loops, MMIO exit classification. **Defect #57: the R1 assertion trips roughly 1 in 6 runs under race.** This is why the QEMU adapter has not been started — building a hypervisor adapter on a memory plane with a known lock violation under contention means any later hang has two candidate causes instead of one.

5.10 kayfabe-isolate — 1832 lines, **BUILT** traits; no real implementation

The port for the unprivileged host worker: `RmBackend` (12 verbs, all of which must be issuable unprivileged), `Isolate` (two-stage retire, cancellation, quiesce predicate), `IsolateFactory`. `Worker::execute` is the single door to a backend and asserts R1 on entry. A `HostHandle` carries its `IsolateId`, so a cross-isolate handle does not merely fail — it does not type-check.

5.11 kayfabe-trace — 1474 lines, **BUILT** vocabulary; not threaded

The typed `TraceEvent` vocabulary, a `TraceSink` port, one-counter total ordering, perf-budget counters and a projection differential. It exists because the GSP milestone's oracle *is* trace replay, so the replay format had to be defined before the harness that consumes it. Two findings from building it are worth carrying: it had to sit *below* `kayfabe-core` (every plane must be able to emit), so it cannot name `RmEvent` or `ProcId` — the bridge is a trait the owning crate implements with an exhaustive match, so a new variant upstream fails the build until the vocabulary names it. And no `&mut Trace` is threaded through any plane signature yet; the vocabulary is driven only from the conformance suite's seam observer.

5.12 kayfabe-shell — 995 lines, **BUILT** (L1-M2, in progress)

The reactor: an epoll loop, source resolution, the inbox, and the executor's wake. It carries the restated F1 rule — *assert a bound, never an absence* — as a concrete quantity: after 16 consecutive unproductive wakes it refuses with `ReactorFault::UndrainableSource` rather than spinning. That restatement came from the mutation gate: three spin-versus-park mutants survived precisely because “never busy-poll” is an absence, and an absence is not observable.

5.13 kayfabe-vmm — 927 lines, **BUILT** traits only

The hypervisor and display ports: `Vmm` (7 capability groups), `Device`, `Present`. An eighth group — the memory-lock primitive — was *removed* from the trait when it turned out to need no hypervisor cooperation on any

backend, which is a good sign about the seam’s discipline. **Nothing implements** `Device` **in production code**; the only two implementors in the tree are test types.

5.14 kayfabe-completion — 756 lines, **BUILT**

Per-process queues (pending → in-flight → awaiting-ack → ack), a drain-gated delivery plane whose re-post is driven off *the owner’s own poll* (the starvation fix), and mapped-fence arms with the #12 backwards-jump guard. Scored 100% on the mutation gate.

5.15 kayfabe-util — 675 lines, **BUILT**

`IntervalMap` (the address table’s container, loud on overlap, empty and wrap), a *virtual* `Instant` so no wall clock exists anywhere in the core, the `assert_send_sync!` compile-time gate, and the lock witness that invariant R1 asserts against. The witness lives here rather than in `kayfabe-rt` specifically so that `kayfabe-isolate`, which cannot depend on `rt`, can still assert it.

5.16 kayfabe-arch — 757 lines, **BUILT** traits only

The identity vocabulary (`HClient`, `HObject`, `Pdb`, `VChid`, `ClassId`, `GpuVa`, `Gpa`, `GpuId`, `EngineKind`) and the Axis-B port set. It grew by 201 lines at this revision, all of it a new `gsp.rs`: a `GspReg` enumeration and a `GspModel` trait reached through `Arch::gsp()`, which is the seam that keeps every register offset out of `kayfabe-gsp`. The GSP port plan’s own §3.5 had recorded “`kayfabe-arch` has no seam for any of this” as a finding; the seam is the finding being paid off. Still zero production implementations: the only `impl Arch` and `impl GspModel` in the tree are `MockArch`, `GspArch` and `FakeGspModel`, all test types. That remains the standing proof that a real one will need no core edits.

5.17 kayfabe-mmio — 314 lines, table **BUILT**; walker is 41 lines of nothing

`AddressTable` is complete and is the enforcement point for the project’s most-cited directive: forward-populate only, unbind eagerly, and **a miss is a fault** — no reverse resolve, no heuristic pick, no MRU fallback exists anywhere in the codebase.

The walker module is 41 lines containing one trait and one enum, **with no walk function of any kind**. `WalkResult` is constructed nowhere; `FbRead` is implemented nowhere. The module’s own doc comment says the walk algorithm is “also here”. It is not.

tests/ — the conformance suite

52 931 lines of test code against 45 421 lines of implementation. There are 30 files in `tests/tests/` and 6 more under `crates/*/tests/`, plus a Scenario DSL in `tests/src/`. The suite is a workspace member with a `[lib]`, which is why `kayfabe-mocks` compiles as a normal dependency. **It grew by 9 659 lines in twelve commits** — the largest single addition being `tests/tests/rmrpc_bridge.rs` (4 474 lines, 65 tests), which is larger than the crate it tests by a factor of four.

That ratio is deliberate and the reason is worth stating, because it is the crate’s answer to an oracle bug found the same week. Every bridge message is transcribed **three independent ways**: an `ogkm`-derived builder in `tests/src/rpcwire.rs` that imports nothing from the decoder, hand-written offset-annotated hex arrays, and the decoders themselves — plus a differential against the C artifact’s independently-transcribed offsets. The bug that motivated it was in `tests/src/gspworld.rs`: the independent guest model derived the element count *from the element under test* instead of reading the field, so the encoder’s own write was unobserved by its own oracle. 250b78c fixed the model; `gspworld.rs` is now 1 468 lines.

6 The invariant catalogue

These are the properties the design is willing to be judged on. Where an invariant has a mechanism, the mechanism is named; where it is only prose, that is stated, because the project’s own doctrine is “*prefer a mechanism to a sentence — a rule stated only in prose has no reader.*”

Invariant	Statement	Enforced by
Order-independence	Everything derived is a pure function of declared protocol facts, never of arrival order. “Protocol, not trace.”	Permutation/interleave/dup proptests over the whole observable end state (<code>tests/tests/determinism.rs</code>).

Invariant	Statement	Enforced by
MISS = FAULT	Tables are forward-populated only; a lookup that misses is a loud fault, never a guess. Refined: a ~28-site audit found the absolute had a second, correct answer — the real rule is decided <i>per site</i> : <i>not yet knowable</i> $\Rightarrow$ DEFER; <i>never knowable</i> $\Rightarrow$ FAULT. A mapping arriving before its PDB bind defers; a handle resolving to the wrong <i>kind</i> faults. Getting it wrong is asymmetric: fault-when-it-should-defer hangs the guest, defer-when-it-should-fault is a security question.	AddressTable has no reverse-resolve path to call. tests/tests/miss_taxonomy.rs makes the taxonomy executable. <i>No shipped site needed a behaviour change; three doc claims did, two of them stating the literal opposite of their own code.</i>
Per-target keying	Pdb and VChid are per-GPU namespaces. Cross-GPU identical ids are legal; same-target duplicates are loud collisions.	project(); tests/tests/multi_gpu.rs.
Per-Proc isolation (I1)	Identical guest VAs and handles in two processes reach disjoint arenas, disjoint host address spaces and disjoint isolates <i>by construction</i> .	Types (GpaSpace::release by value; HostHandle carries its IsolateId) plus a proptest against an injective oracle.
One gated ring path	A doorbell can only reach the host after the #14 working-set gate.	Now structural, not a call-graph claim. VerbPlan::Doorbell is #[non_exhaustive], so its only constructor is VerbPlan::gated_doorbell, which runs the gate; an ungated one is compile error E0639, pinned by a trybuild row. The cardinality claim corrected in §10 is what this replaced.
GSP-S1	We may never emit an element run wider than the guest's staging buffer. At driver 580 the guest reads our elemCount field directly into an unbounded portMemCopy loop whose staging area is exactly queueElementSizeMax / queueElementSizeMin elements — 16 on the bench — with the live msgq metadata as the very next byte. An elemCount of 17 overwrites the metadata the guest is actively using; at the ring's reachable maximum it writes 188 416 bytes past a portMemAllocNonPaged allocation. This is a <i>guest kernel heap</i> corruption reachable from a value we choose.	★ PAID OFF AT THIS REVISION (250b78c), and it was the item the previous revision named as this project's highest-stakes prose-only rule. max_elements is derived from the layout, encode_message refuses GspFault::ElementCountOutOfRange before building a run, <i>unconditionally</i> rather than gated on the layout's field, and the receive side range-checks the guest's declared count and cross-checks it against the derivation (ElementCountMismatch), leaving the cursor unmoved on refusal. <i>With one honest footnote:</i> the originally briefed bite could not fire, because at a dividing element size the length bound already implies the count bound — the count bound is independently reachable only at a non-dividing msgSize, which is real guest-chosen input, and the test moved there.
Completion integrity (I2)	A completion fires only from a genuinely-armed source in the <i>owning</i> process, at most once; the system forge path is typed so it cannot reach a user process's queue.	Type-level (Traffic::System) plus a differential against an independent reference fence model.

Invariant	Statement	Enforced by
RefCount soundness (I3)	A resource is alive $\iff$ it has a live handle or map reference. No premature destroy, no leak, dup survives origin free.	Proptest over deep trees with cross-client dups and interior frees.
DoS containment (I4)	Gpu::apply is transactional; every guest-growable table is capacity-bounded with a loud refusal, never OOM.	Corrected, and the correction is in the design doc: "no $O(n^2)$ path on hostile floods" is false and was measured . The caps bound <i>memory</i> , not <i>time</i> — see §8.
R1	No blocking host verb may be issued while any ranked lock is held.	assert_lock_free at Worker::execute and Worker::with_rm — the only two doors to a backend. <i>It does not guard Drop</i> , which is a real call site, and one R1 violation found this week was exactly an Arc ownership issue in a Drop.
R3	Locks are acquired in strictly increasing rank, at most one per rank.	check_acquire, always-on, panics before the OS acquire.
R5	After re-acquiring, re-validate everything the plan assumed.	The four commit_* functions; bounded re-plan at MAX_COMMIT_RETRIES = 8.
F1	Three unrelated things are called F1 in this project: a fixed threat-model bug, the per-GPU collision guard, and the reactor's bounded-poll rule.	All three real; the overloading is a documentation hazard, not a code one.
GL1–GL13	The guest-memory-lock invariants.	Documentation only. Zero occurrences in any .rs file. They have no port to be enforced at, because capability group 8 was removed from the Vmm trait. Treat as a designed, unimplemented subsystem.

6.1 The two-layer trust model

One normative rule deserves its own space, because it decides designs and because it is the cleanest statement the project has produced:

Layer 1 (trap + lock) gives SECURITY against a guest that ignores the protocol. Layer 2 (NVIDIA's own synchronisation points) gives CORRECTNESS for a guest that follows it. Shared tables are NEVER authoritative for an immediate decision.

[docs/design/core_security_threat_model.md §2.1]

Neither layer is a weaker version of the other. Layer 2 alone is not security: a hostile guest can update a shared table *after* validation and *before* we act, and it can fake the handshake entirely. Layer 1 alone is not correctness: a trap says the bytes cannot change while you look; it does not say they are *finished*. A cooperating guest halfway through publishing a page-directory update has written something internally inconsistent and perfectly stable, and reading it under a perfect lock yields a perfectly-locked wrong answer.

The consequence — *a shared table is evidence, never authority* — is why the address table is forward-populated and why a miss is a fault.

And the C's measurement sharpened it in a way worth carrying. The Layer-2 edge is *whichever* edge the protocol commits at, not a favourite one. On the kernel/UVM/RM paths that edge is the TLB invalidate. On the GSP-emulated compute path it is *not*: round 6 of the #14 investigation measured **zero** occurrences of both invalidate transports there. A design that had encoded "read at the invalidate" as *the* Layer-2 rule would have had no synchronisation point at all on the path that matters most. The general rule survived; one of its instances did not.

What the C artifact does not have is Layer 1. There is no lock of any kind between QEMU and an isolate in a system that runs 22 real GPU applications at host parity. It ships a copy-once snapshot instead, and the reasoning is written out in its source. The working system to date has been running on Layer 2 alone — which is correct for a cooperating guest and is not a security property.

And the boundary points both ways, which building the GSP made concrete. Everything above is about not trusting the guest. **GSP-S1** is the mirror: the guest kernel is inside the boundary we are defending, and the faked GSP is a device that can corrupt guest kernel memory from a width *we* choose. The mechanism is fully traced in `mode2_gsp_port_plan.md` §4.6 and reduces to one sentence — at driver 580 the guest reads our `elemCount` field into a copy loop bounded by nothing but the ring, and the byte after its staging area is the `msgq` metadata it is actively using, so the corruption is immediately self-amplifying. It is the same defect class as the C artifact’s ungarded `% s->q_msgcount SIGFPE`, pointed at the guest instead of at QEMU. The obvious objection — “*the guest is the victim, so it is the guest’s bug*” — is answered in the plan and the answer is that it is both. **At the previous revision this invariant was written down and not enforced, and this paper called that its most security-relevant sentence-rather-than- mechanism. It is now a mechanism** (250b78c; see the GSP-S1 row above and §8.4 for why the bite that proved it is not the bite that was predicted). The general point survives the specific repair: the boundary points both ways, and a faked device is a device.

7 What is tested, how, and what that does and does not prove

7.1 The shape of the suite

Measurement	Value	How
Tests reported green	651	KAYFABE_NO_KVM=1 KAYFABE_SLOW=1 cargo test --workspace, recorded in commit b612a91 — [unverified] here, since no cargo was run for this revision either. Was 547.
#[test.] attribute sites	650	grep over workspace members; excludes the detached generator and the trybuild ui fixtures
proptest! blocks	16	each is one test that runs many cases
Checked-in proptest failures	20 seeds, 4 files	★ This paper said “0” at the previous revision and that was wrong there too — the four *.proptest-regressions files already held 18 seeds at 6ce8599 (§10 row 25). Two were added by 1019064 and pinned deliberately, because that bug had been found once and lost once already
KVM-gated tests	36	CI floor asserts ≥ 35 were <i>reached</i>
TSan targets	4 files, 73 tests	nightly only
Fuzz targets	1	parse_pushbuffer; CI compiles it, does not run it
Implementation lines	45 421	crates/*/src/**/*.*rs
Test lines	52 931	tests/, crates/*/tests/, fuzz/

The suite runs in about 20 seconds with no GPU, no OS dependency and no wall clock [\[unverified\]](#). That is a deliberate and load-bearing property, not a convenience: it is what makes the adversarial suites affordable enough to run on every change. *The one measured timing this revision produced argues the figure has aged:* 92ae2e1 records the workspace suite at **201 s** on a 25-core box with KAYFABE_SLOW=1, which is the configuration CI’s nightly and mutation jobs use.

7.2 The six CI gates

There is one workflow file with six jobs, and the `stable` job grew from twelve steps to **fourteen** at this revision. The two new ones are a *host-pointer* gate (no `*mut`, `*const`, `NonNull` or `transmute` outside a `*_unsafe.rs` file) and a *bridge-exclusivity* gate — no crate but `kayfabe-rmrpc` may name both `RpcCommand` and `RmEvent` in `crates/*/src`.

The bridge gate fired on its first run, on `kayfabe-gsp`’s own stale doc paragraph claiming the bridge would live in `kayfabe-fwd`. That is worth more than the gate itself: a gate that has never been observed to fire is not evidence, and this one earned its keep before it was five minutes old.

Job	When	What it asserts
stable	every push + PR	Fourteen steps: build; test with KVM forced absent ; assert the KVM-gated tests were <i>reached</i> ; clippy -D warnings; fmt --check; the hexagonal-boundary grep; the VMM-vocabulary grep; the generation-name grep (no concrete chip or driver version in the ten logic crates — the gate that makes kayfabe-gsp’s version-agnosticism mechanical rather than aspirational); the unsafe-surface ls gate; the host-pointer gate ; the GPA-accessor gate (exactly one call site — and kayfabe-gsp is on its list, which is why that crate has a GuestRam port of its own instead of touching Vmm); the bridge-exclusivity gate ; three unsafe-containment asserts including the ratchet at 37; and fmt --check for the fuzz workspace.
aarch64	every push + PR	cargo check --workspace --target aarch64-unknown-linux-gnu
nightly-fuzz	every push + PR	the fuzz harness <i>compiles</i>
slow	nightly	the full suite with KAYFABE_SLOW=1
tsan	nightly	ThreadSanitizer over four threaded targets, -Zbuild-std so std’s own synchronisation is instrumented
mutants	weekly	cargo mutants over every production crate

7.3 Exercised versus merely present — the project’s own core doctrine

This distinction is the most useful thing in the repository, and it is worth reproducing the incident that generated it, because it is the template for every criticism in §8.

The conservation ledger’s first census over the mean run reported **0 leaked objects** — and that was a **true negative, not a pass**. A design document claimed the workload “exercises T0 thousands of times per run”; the churn it actually drives adds and removes *RPC* bindings, which own nothing host-side, and the script’s two real subset-frees targeted channels that phase 0 deliberately leaves virgin. **The path had never been reached**. Reaching it took a new test phase. Once reached, the honest baseline was **24 objects, 6 mappings and 24 576 GPA bytes leaking, linear in frees**. [docs/design/testing_doctrine.md §1]

Three rules follow, and the project applies them: a reclamation test that does not first allocate proves nothing; every instrument needs a non-vacuity assertion (a case where it is known to read non-zero); and **bite-check the instrument, not only the fix** — reverting the fix must move the number, and if it does not, the number is not measuring the fix.

Bite-checking is real practice rather than aspiration, and the evidence is that the repository records bite-checks that *did not bite*: at least six such notes are written into the source at the site they concern, each explaining what the neutering failed to move and what was added as a result. One has been promoted into a permanent test that plants a gap and a repeat in a trace and requires the order checker to fire. A commit message from this week reads, in part, “*the bite harness itself lied twice*.”

The doctrine also produces two mechanisms directly. Slow tests are gated by KAYFABE_SLOW, and when unset they print SKIPPED (slow): ... **straight to stderr**, bypassing libtest’s capture; nothing in the suite is #[ignore], on the argument that #[ignore] is invisible in a summary and impossible to count. And the KVM gate prints on *both* arms — KVM-GATE: RAN OR KVM-GATE: SKIPPED — with CI asserting a floor on the sum, because the reverse failure (all 36 gated tests silently vanishing while CI stays green) is equally silent.

7.4 The mean test, and composed versus isolated runs

tests/tests/l1_mean.rs is the largest single test file (35 tests) and is designated *the arbiter*. Its standing is: *pass = the design survived contact; fail = the doc changes, not the assert*. The rule it exists to enforce is an owner directive with four obligations — drive realistic multi-process × multi-thread × multi-GPU traffic end to end through the mocks and assert the observable end state; make it mean rather than happy-path (free mid-flight, race a re-declaration against a still-publishing generation, kill a worker partway, run both lock modes); wire every new milestone into it; and **never narrow a test to make it pass**.

The justification is measured, not asserted. One identity round landed 7 bite-checks; **none of the 363 pre-existing tests caught any of them**. Every real defect that day came from a composed run or a newly sharpened criterion. In the doctrine’s words: “*the happy path found nothing, all day*.”

A representative finding: a test named worker_death_retires_the_proc_loudly_and_never_resurrects was green — and green *only because it never issued an apply after the HUP*. The composed run put a worker hangup and an alloc/map-heavy workload in the same run and found that an out-of-band retire is undone by the next

refresh: a guest able to crash its isolate worker got a **clean new isolate** on its next RM event. The first fix for the related leak class was a use-after-free of its own making.

7.5 The conservation ledger

The instrument all eight teardown paths are measured by. HostLedger replays the recorded verb log and demands every acquisition matched by exactly one release and nothing released that was never acquired — objects *and* mappings. A test may declare a ResidueClaim for what it knowingly leaves outstanding, and the counts are compared **for equality, not as a bound**: *less* residue than declared also fails, so a fix that closes a leak must delete the claim that excused it. There is deliberately no `allow_leaks` escape hatch.

Its stated limit, recorded in the source: the ledger increments only on *success*, so it says nothing about failed verbs. That is why a separate VerbFailure{err, orphans} type exists to carry handles out of a chain that died partway.

7.6 The mutation gate — four lies deep, and still no threshold

The gate mutates the code and measures whether the suite notices. It has now been the most valuable instrument in the project *and* silently wrong **four** separate times, every one of them wrong *upward*. That asymmetry is not coincidence and it is the single most important thing to understand about this instrument: every way an instrument like this fails makes the code look better than it is, because a mutant that is not run, not built or not counted is a mutant that never survives.

Lie 1 — it tested nothing. `kayfabe-fwd` and `kayfabe-rt` have no unit tests of their own; without `--test-workspace true`, cargo-mutants ran an empty suite and reported everything missed. (This one is the exception that proves the rule — it failed *downward*, and was therefore noticed immediately.)

Lie 2 — compiler crashes vanished from the denominator. cargo-mutants rewrites one file per mutant in a copied tree that reuses one incremental dep-graph cache, and that cache does not survive the churn — rustc ICEs. cargo-mutants cannot tell a compiler crash from a type error, so it files the mutant *unviable* and **silently removes it from the denominator**. Measured: **136 of 293 mutant builds ICE'd**, turning a real 88.95% into a reported 67%. CI now fails on any ICE in a mutant log, and on a zero viable count, *before* the threshold is read.

Lies 3 and 4 were found at this revision, are worse than either of the first two, and are covered in §8.4: the configuration file was never read, and the gate could report 100% from a broken suite.

The historical numbers, each true of the scope it was measured on: **99.2%** (245/247) on the pure L0 core; **92.44%** (159/172) on the L1 threaded surface. Both were measured over a population that *included the instruments*, because the exclusion file was inert.

Do not quote a mutation threshold. There is deliberately not one to quote. The CI value is still 91, and the workflow marks it **PENDING RE-DERIVATION** in its own text: 91 was derived against a four-path scope covering roughly 41% of the implementation, and the scope is now every production crate. The project's own rule is that the bar is derived from a measured baseline under the configuration CI will actually use, and no such run has completed. A clean campaign is in flight; **its number is not available to this document**.

The campaign that *has* completed, labelled honestly. 2026-07-28, 2 898 mutants, 5 h 49 m: 2 305 caught, 112 missed, 3 timeout, 478 unviable. Product-only, with scaffolding removed from the denominator, that computes to **95.52%** (**95.38%** if a `TIMEOUT` is not counted as a kill, which is a real choice — cargo-mutants scores a timeout as *killed*). Δ **That run's tree was modified underneath it while it ran**, by a concurrently dispatched task, and the number is therefore reported as an indication and not as a measurement. It also does not describe the population CI will now test: with the configuration finally being read, `--list` yields 2 667 targets, not 2 898.

What that campaign found, which is worth more than its score. `kayfabe-gsp` is the floor at **92.71%** and holds 36 of the survivors — exactly as expected for a crate no campaign had ever covered, and the direct answer to the previous revision's "no mutation campaign has ever run over them". `kayfabe-abi`'s encode side, previously a named gap, is **closed**: 195 mutants, zero survivors. Of the 115 survivors, **8** are provably equivalent (each proven by its sibling mutant being caught at the same site), **22** are dead public API or Debug diagnostics with no callers anywhere, **17** are scaffolding, and the remaining **68 are real untested logic, individually named**. A debt that is enumerated to 68 named items is a different object from a percentage.

And one methodological result from closing some of them, which cuts against the usual reading of a surviving mutant. `b612a91` closed twelve `kayfabe-gsp` survivors and found that one of them — `a + → *` in `RegionMap::load` — was not guarding correct code. It was *masking wrong* code: the function reported `pages.len()` + `i` as the index of a misaligned page-table entry while `pages.push` ran in the same loop, so the reported index was $N + 2i$ and correct only for the first entry of each table page. A guest-facing diagnostic naming the

wrong entry of a hostile table. **"This mutant survived" is therefore not automatically "write a test"; it is sometimes "read the function".**

Three of the targeted survivors were *deliberately not closed*: `RegionMap::is_empty` and `page_size` have zero callers repo-wide, and writing tests for them would be exactly the score-gaming the instrument exists to detect. `is_empty` cannot simply be deleted (`clippy::len_without_is_empty` requires it once `len` exists) and deleting dead public API is an owner call. That distinction — kill what a real readback naturally asserts, refuse to invent a probe for what nothing calls — is the whole difference between a mutation score and a coverage number.

7.7 The OS-free configuration, and why it exists

`KAYFABE_NO_KVM=1` forces the KVM probe to report the device absent, so the pure logic core plus mocks — no OS layer at all — runs on purpose on any box. CI runs *this* as its default test configuration.

The reason it was made explicit is the doctrine applied to itself. GitHub runners have no `/dev/kvm`, so CI was already running the OS-free configuration — **by luck of what the runner image happens to lack**. It was unreproducible on any developer box, and it would have lapsed silently the day a runner image started shipping the device. In the commit's own words: *"a guarantee that holds by luck is the same shape as a green instrument on an unexercised path."*

The owner's question that prompted it is worth repeating because it is the right question: *"I hope you didn't delete the mocks so the core can still be tested without OS layer? and that must pass as well so we don't lock in architectures/os semantics/qemu and later regret."*

7.8 What the suite does not prove

- **It does not prove anything about NVIDIA.** Every test runs against `MockArch`, whose encodings are deliberately *not* NVIDIA's. That is the right design for proving the seam, and it means the suite cannot catch a wrong belief about the real protocol. Only the C artifact and the bench can do that.
- **The with-KVM configuration has no automated gate.** CI runs *only* the KVM-absent configuration. The 36 tests that use a real `/dev/kvm` — including the entire real memory plane and the vCPU/MMIO dispatch — run on developer boxes and the bench, and nowhere else. This is the largest hole in the CI story, and the workflow says so in its own text rather than hiding it.
- **The fuzzer is compiled, not run, by CI.** One target, over the one place raw guest bytes reach the core. Its two historical findings were minimised into deterministic in-tree regressions, which is the stronger form. The corpus is git-ignored, so a fresh clone starts from empty.
- **Single-threaded logic testing of a concurrent system.** The core is tested deterministically because it is a pure state machine; the concurrency is tested separately by the threaded suites plus TSan. The argument is sound, but it depends on the core genuinely having no interior mutability — which is asserted at compile time, and is the sort of property worth re-checking if you are looking for holes.
- **The page-size axis is designed and not built.** The testing doctrine cites [4 KiB, 16 KiB, 64 KiB] as an application of its "both modes ship from day one" rule. The environment variable that would select it does not exist in any `.rs` file — only in forward-looking comments. The `LockMode` half of that same rule *is* built and exercised.

8 Discussion: the good, the bad, and the risky

8.1 What is genuinely good

The purity constraint is real and it is mechanised. A "pure core" claim is usually aspirational. Here it is three CI greps, a manifest-level `forbid`, a naming rule that makes `1s` the audit tool, and a ratchet on the exact number of unsafe relaxations. The external dependency surface is three crates, two of them dev-only.

The failure ledger is unusually honest and it is load-bearing. Two design documents carry "contact logs" — `11_concurrency.md` §12 and `11_os_shell.md` §14 — recording every place building the code proved the design wrong. They are long. They contain entries whose titles are things like "the criterion was wrong", "boundary-1 broken", "two sections of §3 contradict each other", "the design was wrong about its own severest claim". A project that writes these down is a project whose other claims are worth more.

The adversarial suites found real bugs before hardware existed. 15 at L0, more since at L1, including two control-plane *wedge* bugs reachable by a hostile guest process — a parked `SET_PAGE_DIRECTORY` and a parked `MAP_MEMORY_DMA`, each of which could be aimed via a dup alias to make a benign third party's allocations fail forever. Both were found by a property test, not by inspection. *The same parked-fact machinery produced this revision's worst core defect too* (§8.3), which is a fair reading of where the difficulty in this design actually lives.

The bridge was designed on paper before it was built, and the design was then reported against. `gsp_core_bridge.md` was written first, in full, including the finding that changed the roadmap; the four implementation commits then recorded roughly two dozen places where that design met the compiler and lost, *by section number*, including three of its own test obligations turning out to be unbuildable and one turning out to be wrong. A design document that is graded by the code that implements it is worth more than one that is merely followed.

Both lock modes and both fallbacks ship as first-class tested modes. The argument is sharp: the slow path is the path the system takes *when something has already gone wrong*, and a fallback exercised only by the fast path declining has test coverage proportional to how often the fast path fails — which is the quantity the optimisation exists to drive to zero.

The ABI oracle design. Pinning generated layouts against three independent readings, with a test that documents exactly how they disagree, is better than what most projects do, and it caught the C artifact's own parity test being weaker than the code it guarded.

8.2 ★ The bridge exists — and it does *not* unlock compute

This section replaces the previous revision's headline, which was: "the critical path is the bridge, not the crate." The bridge is built. It was designed first (`docs/design/gsp_core_bridge.md`, 865 lines) and then built in four stages, each independently green, each recording where the design met the compiler. If you only read one thing about it, read §2.7 of that design document, which is the finding that changed the roadmap.

What was built

`translate(&DriverAbiTable, &RpcCommand)` is a stateless free function; `GraphPolicy` is the `CommandPolicy` the boot FSM calls, doing `translate → Gpu::apply → reply`; `kayfabe-abi` gained the per-class alloc table and four params decoders; and the control arm (function 76) decodes what it can model and **refuses the rest by name**. The refusal set is fifteen variants wide and every one of them carries the number that caused it, because the census is the stage's real output: it says which decoder is missing, not merely that one is.

★★ `MAP_MEMORY_DMA` is not on the wire, so the bridge cannot deliver compute

This is the finding to carry away, and it is a roadmap fact rather than a code fact.

On a GSP-client GA106, `rpcMapMemoryDma` (function 14) and `rpcUnmapMemoryDma` (function 15) are **_STUB through the entire HAL chain** — `_GA106 → _GA102 → _GA100` — and the stub returns `RPC_UNKNOWN_FUNCTION`. The real implementation is installed only by the *vGPU* IP-version table. So `RmEvent::MapMemoryDma` **has no producer on this path, and no stage of this bridge builds an NVOS46 decoder**.

Three independent oracles agree, which is why this is stated flatly: the open kernel modules' own generated HAL tables; the C artifact, which booted a real GA106 guest to CUDA and has **no function-14 hook at all** in its complete snoop set (47/76/21/103/10/65/70/72/73); and two C-era design documents that say it in words — "there is no per-map GSP-RPC carrying *VA↔phys*".

The consequence, stated so nobody reads the bridge as one stage from compute. The address table's populate sources are the `GPU_PROMOTE_CTX` control and **CE page-table-write capture** — and both are *kayfabe-fwd* / *kayfabe-mmio* work in different crates, neither of which is built. `GPU_PROMOTE_CTX` is not even shaped like an `RmEvent`: it declares a *set* of (`gpuPhysAddr`, `gpuVirtAddr`, `size`, `bufferId`) entries, which is an address-table population rather than an object-model edge, so it does not belong in this seam and was deliberately left out of it. Finishing the bridge to B6 would give a complete object model from wire bytes and **still** leave the address plane empty. The compute path is not one stage away; it is a different plane.

★ `SET_PAGE_DIRECTORY` is not how an ordinary address space declares its root

B4 was built to decode the control that carries a page-directory base. It settled the question *against* the design that asked it.

On a bare-metal GSP client, `SET_PAGE_DIRECTORY` reaches the wire **only** for an address space marked `SHARED_MANAGEMENT` or `IS_EXTERNALLY_OWNED` — `gvaspaceExternalRootDirCommit_IMPL` asserts on exactly that, and its only callers are the UVM / gpu-ops path. Every *ordinary* RM-managed VAS declares its root through `NV90F1_CTRL_CMD_`

VASPACE_COPY_SERVER_RESERVED_PDES (0x90f10106) at construct time, as `levels[0].physAddress`, on a path that is on by default for any GSP client. There is a third: `NV2080_CTRL_CMD_INTERNAL_GMMU_COPY_RESERVED_SPLIT_GVASPACE_PDES_TO_SERVER`.

The design's arm was therefore **necessary and not sufficient**. All three commands are refused as a *distinct* variant, `PageDirControlNotModelled{cmd}`, rather than folded into the generic unknown-control refusal — and the reason is the sharpest small argument in the bridge: **a dropped page-directory declaration is invisible**. The VAS never routes, every channel defers at its first doorbell forever, and nothing anywhere says why. A separate refusal variant converts a silent hang into a counted, named record of exactly which decoder the address plane is waiting on.

Two smaller results from the same arm are worth keeping because they are the kind of thing a version table does not catch. `hVASpace == 0` is **not** "unspecified": NVIDIA's own header says verbatim, in both vendored trees, that zero means *the implicit VA space associated with the client/device pair* — a real object the RPC does not name and the graph has no node for, so passing it through would park a PDB on a node the guest never declared. It is refused. And note the deliberate asymmetry: on the *alloc* arm, a zero handle means "nothing declared" and maps to `None`. Same zero, opposite meaning, documented differently by NVIDIA — so neither may be inferred from the other.

Where the design met the compiler

Roughly two dozen items are recorded across the four commit messages, and the pattern in them is more useful than any single one: the design's *taxonomy* claims survived and its *buildability* claims did not. "Known-and-inert versus unknown, there is no third state" was contradicted by the first stage's own row — there are at least four states. Three separate non-vacuity arms the test plan specified turned out to be unbuildable at the stage that specified them, one of them because it was also *wrong*: refusing a second, different PDB for one VASpace requires remembering the first, which a stateless bridge cannot do and which `RmGraph` had already decided the other way, with an argument (re-binding is protocol-legal; refusing would hang a conforming guest).

One item is a finding the design does not have and could not have had: **functions 72 and 73 never reach the policy at all**. The FSM returns on a no-reply disposition *before* calling `respond`, so those arms are unreachable through this adapter. Harmless today, because neither carries object-model content — and load-bearing to know, because a future no-reply function that *did* carry a fact would be dropped silently.

8.3 ★★ A live-memory-free defect — and the independent oracle had the identical bug

This is the most serious defect the project has found in its own core, and the part that matters is not the defect.

The defect, one line wide. `RmGraph::apply` drained its three parked-fact tables from the `Alloc` arm only, on the reading that a parked fact waits for its target's *allocation*. It waits for its target's **handle** — and `Alloc` and `Dup` are the two events that mint one. So a dup-of-a-dup whose middle handle was minted by a `Dup` stayed parked *forever* while being fully resolvable.

The measured consequence. `project()` follows parked dups through `origin_of`, so the process grouping saw the alias; the refcount never did. The undercount takes references to zero, the resource is destroyed, the component vanishes, the process is vacated — and **the reaper emits `Free` for memory a live kernel alias still references**. That is the project's own "never free host memory RM says is live" rule, reached through a door nobody was watching. It was three defects, not one: the same missing drain also left `Resource::pdb` unset for a parked page-directory declaration and never installed a parked mapping, giving a fault on a *legal* guest.

★★ **And the independent oracle carried the identical bug.** The property suite's `RefTracker` — the deliberately separate reference-count model whose entire purpose is to disagree with the implementation — also promoted parked facts only in its `Alloc` arm. Model and implementation were wrong *the same way*, so the property stayed green over a three-defect bug for as long as both existed. The correction is proven to be a strengthening rather than an assertion: with the model fixed and the implementation's drain removed again, the property fails on its own.

Why this is the most instructive entry in the paper. An independent oracle is this project's strongest single testing idea — it is what `kayfabe-abi`'s three transcriptions are, what `gspworld` is, and what the bridge's hand-hex fixtures are. This is the failure mode of that idea, in its purest form: *independently written* is not *independently reasoned*. The same author, holding the same wrong sentence about what a parked fact waits for, wrote it into both. Nothing about the shape of the test caught it; what caught it was a different invariant, derived from the other side of the graph, disagreeing.

8.4 ★★ The quality instruments were wrong — three of them, in one day

This is the real theme of the work since the last pin. The code got better; what got *much* better is the confidence one is entitled to have in the instruments that say so. Each of the three below was a live, silent, one-directional error in the machinery this document quotes.

1. The mutation configuration was never read. `mutants.toml` sat at the workspace root. `cargo-mutants` reads `.cargo/mutants.toml`. **The file was never opened, and every exclusion argued inside it was inert** — including the exclusion of `kayfabe-mocks` and `tests/src`, which is the one this paper has repeatedly relied on when saying “excluding scaffolding”.

It was proven rather than inferred, which is the right standard for a claim of this shape: appending a bogus key to the root file changed nothing and produced no parse error; appending the same key to `.cargo/mutants.toml` fails loudly. `cargo mutants --list` goes from **3 760** targets to **2 667**, and scaffolding mutants listed goes from **361** to **0**.

The consequence for every number this document has ever quoted. Every mutation figure recorded before 2026-07-28 was measured over a population that *included the instruments*, and the “*~94.9%* excluding scaffolding” figure — which this paper printed at the previous revision — was a **hand computation**, not something the tool ever did. That is exactly why it read as a parenthetical rather than as the gate: it was never the gate.

2. The gate could report 100% from a broken suite. `cargo-mutants` baselines with `--package=` naming only the mutated packages, while the mutants themselves run `--workspace`. Measured: the baseline runs about 200 tests across 39 binaries and **omits** `kayfabe-tests` **entirely** — the conformance suite that catches most mutants. A red conformance suite would therefore pass the baseline and then make *every* mutant look caught. Fixed with needs: `stable`, which makes the real suite a precondition of the gate rather than an assumption of it.

The same job’s `--timeout 240` sat at or below honest completion (the workspace suite alone is 201 s on a 25-core box, and a 4-vCPU runner at `-j 2` is slower). **cargo-mutants scores a TIMEOUT as killed**, so mass timeouts would have reported near-100% and the gate would never have fired. Raised to an absolute 1 800 s — and deliberately *not* to a `--timeout-multiplier`, because the multiplier is computed off the same baseline that omits the suite, and would reintroduce the identical inflation through a different door.

3. Thirteen test park-points hung forever instead of failing, producing zero output. The shape is a `thread::scope` whose cleanup is reached only by statements *after* the assertions: any failed `assert` unwinds past them, and the scope blocks forever joining threads that were never told to stop.

It had already happened, for twenty-three hours. Three wedged test binaries were found alive on the box, three threads each, all parked in `futex_do_wait`, at *~83 000 s*. And the failure mode is worse than “buried”: under CI-realistic output capture **the panic message is never emitted at all** — the log stops at running 1 test and stays empty forever. One site hung *spinning*, burning a full core, and outlived the cargo process that spawned it.

Every one of the thirteen was **proven vulnerable before and proven fixed after by inducing a real failure at each site**, not by reading the code — which is the same bite-check discipline the project applies to its assertions, applied to its harness. The fixes are `Drop` guards armed as the first statement inside each scope closure, plus a barrier type that panics instead of blocking when its rendezvous breaks. Each guard carries an in-comment R1 argument naming its callee, because *a Drop is a call site* and R1 does not guard `Drop`.

And a fourth, smaller, which is the same species. The test count itself was wrong: 551 came from **stale prebuilt binaries** in `target/`. Two test files had 16 and 32 `#[test]`s at HEAD while the inherited cache ran 13 and 24. A related trap bit twice in one day inside the bite harnesses: `cp -a` and `shutil.move` both preserve `mtimes`, so a restored file keeps the previous mutation’s compiled artifact and a whole bite table comes back *inflated*. Any harness that restores files must touch them.

What these four have in common, and why they belong in a whitepaper rather than a changelog. Every one of them made the project look *better* than it was, and every one was invisible from the output. An inert config, an omitted baseline, a timeout scored as a kill, a hang that prints nothing, and a stale artifact are all the same failure: **the instrument reported a number that no longer described a measurement**. This is the project’s own founding doctrine — “*a green instrument on an unexercised path is worse than no instrument*” — turned around and pointed at

the instruments themselves, and it is the strongest available evidence that the doctrine is being applied rather than quoted.

8.5 kayfabegsp: boot is modelled, reboot is not, and one open item has no proposed resolution

Two revisions ago this section said “34 lines, and it is the critical path”. One revision ago it said “3 550 lines, and nothing calls it”. Both are now superseded: the crate is 3 828 lines across 8 modules, it has a production consumer (§8.2), and the one invariant this paper flagged as prose-only — GSP-S1 — has been paid off in code. What has not changed is everything below. Three of the plan’s nine stages are unbuilt and all three are the ones that need a GPU; one open item still has no proposed resolution; and the plan the crate was built from was written against the wrong driver version, a fact that keeps producing new consequences rather than being absorbed once.

What is built, and what “built” is worth here

The five seams the crate claims — GPU architecture, driver version, hypervisor, GPU lifetime, CPU architecture — each have a test that pushes on them, which is the difference between a seam and a decoration. Two are worth singling out because they are cheap to fake and were not faked. The lifetime seam is exercised by running *repeated* driver lives in one process: the C artifact’s measured second-life failure (`msgqRxLink` timeout, -7, 71 064 retries) has exactly one cause in the driver’s own source, `rx.size != size`, and the FSM is shaped so that cause cannot arise. The architecture seam is exercised by driving the same FSM through *two* deliberately-different register models, which is the same argument as `MockArch` one layer down.

The oracle is the part to be sceptical of, and the crate says so itself. The design called for trace replay against the C emulator’s recorded boots. Those traces do not exist: the capture patch is ~60 lines against a C file this work does not own, and it has not landed. So S5 is proven against a *guest model* — an independent implementation of the driver’s receive path — which is a stronger oracle than a synthetic script for everything except the one thing it was supposed to establish. Until the capture lands, “byte-equivalent with the C” is untested.

Boot is modelled. Reboot and resume are not.

The plan has nine stages. S0–S5 — the six that need no GPU — are the boot and steady-state protocol, and they are what `f2055bf` built. S6, S7 and S8 are the three that need a GPU, and none is built; S6 is additionally blocked, because `4a0cb29` records the bench host as offline at the provider. Two of the plan’s open questions (O3, whether the sequence numbers reset across a true `rmmmod/insmod`; O5, which boot mode a post-teardown re-acquire uses) are answerable *only* by a bench run, and O3 is not even reachable before S7. This matters more than the stage count suggests, because the owner’s standing requirement is that a GPU restart — idle, released, re-acquired — must work *with no bolt-on*, and restart is exactly the region S6–S8 covers. Boot is the part that could be modelled offline, so boot is the part that got modelled.

★ The sharpest open item: at 580 the resume handoff is an RPC we would have to send

This is §11-O7a of the port plan, and it is the one item in the whole component with no proposed resolution, because the evidence that would settle it is not in any open-source tree.

The mechanism is present at both driver versions and reads identically: a core resume resets into RISC-V, reprograms the LibOS boot-args address, starts SEC2 and waits on `NV_PGC6_BSI_SECURE_SCRATCH_14._BOOT_STAGE_3_HANDOFF`. What differs is who initiates it:

	580.159.04 (the bench)	610.43.02 (the vendored spec)
where the wait lives	inside the CPU sequencer executor, as opcode <code>CORE_RESUME</code>	promoted to a first-class HAL, <code>kgspExecuteCoreResume_TU102</code>
how it is reached	only through <code>_kgspRpcRunCpuSequencer</code> — i.e. only if we send a <code>GSP_RUN_CPU_SEQUENCER</code> event	locally, from two call sites in the driver. Nothing is required of us.

Three things follow, and none of them is comfortable.

It is not a layout difference, so a version table does not absorb it. Every other 580-vs-610 divergence the project has found is an offset, a field count or a constant — one row in a version-keyed table. This one is a difference in *who initiates*, which means a faked GSP that must emit sequencer buffers at 580 and must not at 610 has version-conditional *behaviour*. docs/design/four_axes_of_variation.md §5 rule 1 forbids exactly that: “No if guest_version == ... in a logic crate. Transitions fire on observed protocol facts, never on driver identity.” The two honest options named in the correction brief are a version-keyed *capability* on the FSM or a loud refusal, and the brief declines to choose between them on the grounds that choosing requires an answer it does not have.

The answer cannot be read out of the open source. What the open tree shows is that the path exists and how it is triggered. What it cannot show is whether the firmware ever triggers it on a path we must support, because the *contents* of a sequencer buffer come from GSP-RM firmware, which is not open. Specifying an emitter now would be guessing, and the plan says so rather than shipping a guess.

So the requirement is genuinely at risk, and this is the cleanest statement of it in the project. If any restart scenario in scope reaches CORE_RESUME at 580, then a faked GSP that never emits GSP_RUN_CPU_SEQUENCER cannot drive it, and “GPU restart must work with no bolt-on” is not currently satisfiable by the design as written. Whether such a scenario exists is unknown [unverified]. The next step is a reading task, not a hardware task — enumerate the callers of consequence of the sequencer executor at 580 — and it is recorded as O7a rather than resolved.

One further sharpening worth carrying, because it cuts against the project’s own instinct: the relayed finding that started this said “580 has a resume handoff 610 never reads”. **That half was wrong** — 610 has the identical routine reading the identical register. The correction log reports it as wrong rather than quietly fixing it, which is the behaviour a reviewer should be checking for.

★ The plan was written against the wrong driver, and one item was RESOLVED backwards

This is the most instructive thing that happened to this component, and it is not flattering.

The port plan took its NVIDIA citations from the vendored open kernel modules at **610.43.02**. **The bench runs 580.159.04**. The two do not merely differ in offsets; they differ behaviourally, and 3 550 lines were built against the un-caveated plan before anyone vendored the matching tree. Eight claims changed when ogkm-580.159.04 was finally vendored (7af23cb): the element count is *read from a field* at 580 and *derived from a length* at 610; MCTP/NVDM transport headers do not exist at all on the 580 GSP path; the layout break interval narrowed from (570, 610] to (595.84, 610.43.02]; the init RPCs are queued *before* bootstrap at 580 rather than inside it; the suspend sentinel is an exact equality at 580 and a mask at 610; and the queue geometry is compile-time at 580 rather than negotiated, which removes the “the guest declares its own geometry” argument the plan had presented as its highest-leverage design choice.

★★ And at this revision the same seam bit inside a range the project claims to support, which is a strictly worse shape than “the plan cited the wrong tree”. NV_CHANNEL_ALLOC_PARAMS **diverges at offset +32**: 610 inserts hHandleVASpace there, where 580 — *the bench driver* — has hUserMemory[0]. A generated 610 mirror would mis-read **every field from +32 onward** for the guest we actually run, engineType included, and would do so *plausibly* rather than loudly.

The response is the one the four-axes rule implies and it costs something: **the struct is deliberately not generated at all**, only the 32-byte agreed prefix is decoded, and a test named the_channel_alloc_prefix_stops_where_the_two_trees_stop_agreeing asserts the *absence* of the generated struct so that nobody helpfully adds it later. What that costs is concrete: engineType lives past the prefix, and **a CUDA process’s GR and CE channels are the same hClass** — so Arch::classify cannot answer which engine a channel is for, and a CE channel only becomes one later, via its engine object and a refinement pass. A version-keyed *table* does not absorb this; only a decoder that declines to decode does.

The worst instance is §11-O7, which had been marked RESOLVED with the answer that is backwards for our bench. It said GSP_RUN_CPU_SEQUENCER “is not implemented, do not emit it” — true at 610, where the executor was deleted; at 580 it is fully implemented, dispatched, and first on the bootup-window allowlist, so an emitted sequencer buffer would execute register writes and falcon resets inside the guest kernel rather than being harmlessly ignored. The RESOLVED status has been withdrawn.

The class, which is the part worth keeping. A *versioned* specification was cited as if it were *the* specification. The fix is mechanical and now normative: every citation carries its tag (ogkm-580: / ogkm-610:), an untagged claim is unverified by definition, and line numbers *and paths* never cross tags — the whole msgq library moved directories between the two.

★ **That fix is being applied to new code and the backlog is getting worse, which is the honest and unflattering measurement.** At the previous pin the crates carried 121 bare, untagged ogkm: citations and zero tagged ones. At this revision there are 56 tagged citations — the rule is real and new work follows it — and the bare count has risen to 159, because four crates’ worth of new code was written faster than the backlog was converted. `kayfabe-gsp`’s own share fell slightly, 96 to 92. **The CI grep that would stop the class recurring is specified and still not written**, and it is now the only thing standing between a normative rule and a growing pile of counter-examples. The task is filed (correction-brief item B11) and was explicitly not reached.

The self-critical note in the correction log is the one to read: one claim in the same document was marked [inferred] and turned out to be right, while O7 was marked RESOLVED and turned out to be backwards — and the difference was not luck. *The inferred claim knew it was second-hand and the resolved one did not.*

What this does and does not change about the critical path

The gap moved up one layer for the second revision running, and it did not close. The bridge exists and is production code, so the previous revision’s answer is spent — but nothing outside `tests/` implements `GuestRam` or `Device`, nothing outside `tests/` constructs a `GraphPolicy`, and the wire-layout selector `DriverAbiTable::gsp_element` now exists with **no caller in any crates/*/src file**. So the version-dispatch machinery that makes the crate version-agnostic still has nothing production-side asking it a question — only the fact that had no key now has a key and no interrogator.

The critical path is now the L2 QEMU adapter (§8.10), which is the thing that would give this whole chain guest RAM to read and a BAR to answer from — and which has not been started, deliberately, because `kayfabe-vmm-kvm` still carries defect #57. *Read that sentence with §8.2 in hand*: even a complete L2 adapter feeding a complete bridge produces a correct object model and an empty address table.

8.6 Nothing has ever touched a real GPU

Stated once more because it is the thing most likely to be forgotten between here and the end of the paper. There is no binary. `Device` has no production implementor. `RmBackend` and `IsolateFactory` have exactly one implementation each and it is the mock. No isolate process has ever been spawned. The composition of threads that would constitute a running system exists only inside one test file.

The faked GSP and its new bridge do not change this, and it is worth being explicit, because they are the components most likely to be misread as hardware contact. `kayfabe-gsp` parses real `msgq` headers, real element framing and real RPC envelopes; `kayfabe-rmrpc` decodes real `NV0000/NV_CHANNEL/NV90F1` layouts out of them — but every byte either has ever seen was written by a test into a `BTreeMap` behind the `GuestRam` port, or hand-typed as an offset-annotated hex array. **No guest driver has ever posted to any of it.** The GSP’s own oracle, trace replay against the C emulator’s recorded boots, is still **not available**: the capture patch has not landed, so the traces do not exist, and the crate is proven against an independently-written model of the guest rather than against the guest — with the caveat that §8.3 is a worked example of an independently-written model being wrong in exactly the same way as its subject. Two of the GSP’s remaining open questions (O3, O5) are answerable only by a bench run.

What this specifically means for the claims in this document: everything about *structure* (the DAG, the purity, the type-level isolation, the lock discipline) is verifiable today and this paper verified it. Everything about *behaviour against NVIDIA* is inherited from the C artifact and re-expressed, and the re-expression has not been checked against the thing it models. The gap between “our mock says the core does the right thing” and “a real driver agrees” is exactly the gap the C artifact spent two months and 739 commits closing, once, in C.

8.7 The quadratic control plane — an open, measured DoS

The invariant catalogue’s I4 says hostile input is *contained*: no wedge, no OOM, no bystander corruption, every hostile event earning only its own refusal. All of that is property-proven. It says nothing about *cost*, and the project measured the cost and recorded the result against itself:

The control plane is $O(\text{live objects})$ per event, so N events cost $O(N^2)$. Measured on a debug build: **1 000 events 0.85 s; 8 000 events 54.8 s**. The capacity cap is 2^{18} live handles, so a guest can drive that curve deliberately — and **PyTorch startup reaches it benignly**. Deleting the per-event graph clone saves ~24% and leaves the curve quadratic, so the clone is neither the cause nor the fix.

The stated statement in `ARCHITECTURE.md` — “no $O(n^2)$ path on hostile floods” — is marked **false** in the same document. Two candidate fixes are named and neither is small: incremental derivation, which is a redesign of the “derived state is a pure function of the graph, never accreted” decision that the entire determinism differential rests on; or an undo journal in the most safety-critical file in the repository. A sibling of the same species *was* fixed (a condemned-list carry-forward, $O(n^2 \log n)$, 55 s $\rightarrow$ 3.8 s, by union-find), which is evidence the class is tractable and also evidence that it recurs.

8.8 Reclamation is the least finished plane

Figure 5 marks two triggers red. Restating the sharper form:

“Retire eager, reap deferred” is currently half a law. The deferral has no bound and nothing arms it. In a run that never quiesces, reap-deferred and reap-never are indistinguishable.

The one genuine cross-process reference has a disposition that no ledger will like. On a kernel/UVM dup of a user address space, refcount zero and per-object `Free` are not the same event: the last reference retires and reaps the owner *without issuing a single `Free` verb*, and the disposition of record is the isolate process’s death. GPA is conserved independently and asserted. But any future reclamation ledger, quota or accounting hook will be wrong on that path first — which the design says, in advance, in its own words.

And the most leak-prone moment has no vocabulary at all until L1-M2 lands. A worker that dies *mid-chain* cannot run its own unwind, so everything allocated before the failure is in no `Orphans` and in no core state. It is unrecoverable and no current test can see it. The design’s answer — escalate to killing the isolate, because killing the process is how you reclaim state you cannot enumerate — is correct and is not yet built.

8.9 arm64 is measured for the core and unmeasured for the shell

The full suite *ran* on a GB10 Grace-Blackwell aarch64 host: **372 passed, 0 failed**, identical to x86_64. That is a real upgrade over a cross-compile check, and it covers the concurrency suites, whose memory-ordering behaviour is exactly what a `cargo check` cannot see.

What it does not establish, stated by the project so nobody over-reads it: `getconf PAGESIZE` was 4096 on that host, so the 16/64 KiB page-size rule — the one real pressure point — is still unexercised. The box was a container with no `/dev/kvm` and no `/dev/userfaultfd`, so **no L1/L2 OS-shell behaviour was exercised on ARM at all**. And there are zero VM-capable arm64 machines on the bench provider, so settling the remaining question by experiment requires hardware from somewhere else.

The honest summary is the project’s own: *the pure core is arm64-clean by measurement; the OS shell on arm64 is entirely unmeasured.*

8.10 The QEMU $\geq$ 10.2 floor buys less than the decision that took it assumed

The design requires `memory_region_enable_lockless_io()`, which lands in QEMU 10.2. That buys a genuinely important thing — MMIO dispatch without the big QEMU lock, which is what makes the doorbell path viable — and upstream’s helper also sets the reentrancy guard in the same body, so a discipline the design planned to maintain is no longer ours.

The floor decision’s second claim was economic: requiring a version rather than patching one deletes “*the whole ‘which QEMU are we built into’ ambiguity*”. **The L2-Q adapter design (b31487a) checked that claim against QEMU’s source and it does not hold.**

QEMU v10.2.0 has no supported out-of-tree device mechanism, so our device must be compiled inside a QEMU source tree. Two mechanisms, both closed: `module_load_qom()` resolves a QOM type name against `module_info`, a table generated at QEMU build time, so a .so that is not in that table can never be discovered by type name — and `QEMU_MODULE_DIR` changes only the directory searched, not the table. `module_load_dso()` additionally requires the module to export a per-build `CONFIG_STAMP` symbol; upstream’s own failure hint is verbatim “*Only modules from the same build can be loaded.*”

[v10.2.0 util/module.c, include/qemu/module.h]

State it exactly, without overstating it. The floor *does* delete a patch we would otherwise have had to carry: the cancelled 9.2 backport modified upstream semantics inside `system/phymem.c`, had to be re-derived every release, and a mis-rebase would silently change how *every* device in the machine dispatched. What replaces it is an additive overlay — a directory dropped into a stock 10.2.x checkout plus two hunks (one `meson.build` line, one `Kconfig` stanza) — whose only conflict surface is a build failure, never a behaviour change. That is a real improvement. **What it does not buy is “install your distro’s QEMU and run”.** The user builds QEMU once. Three of the four costs the floor decision itemised against the backport — a rebase every release, a build script that must reproduce it, and a supply-chain claim we make to every user — still apply.

There is exactly one path to a genuinely stock QEMU, it is named, and it is not taken for v1. `vfio-user` would make our device a separate process speaking a documented socket protocol to an unmodified distro-packaged QEMU. It is present at v10.2.0 (`hw/vfio-user/`, two documentation files, a vendored `libvfio-user`) and MAINTAINERS lists it `S: Supported`. It would delete the QEMU tree, the overlay, and the second unsafe crate the in-tree route needs. It is rejected for v1 on one source read: nothing in `hw/vfio-user/` calls `memory_region_enable_lockless_io`, so a trapped BAR access becomes a socket round trip *taken with the BQL held* — reintroducing and amplifying the exact serialisation the floor was taken to remove. The design records the small experiment that would change the verdict (measure the round trip both ways, and ask whether marking the client's regions lockless is a patch upstream would accept) rather than closing the question.

The facility inventory that priced the floor also **reversed two of its own design assumptions**, which is worth knowing before trusting the rest of it: the read-only-memslot region-lock fallback, priced in the design at 1.49 μ s as "a flags-only flip", is *unreachable through QEMU* — QEMU's flatten pass turns a readonly change into a delete-plus-add. And checkpoint-restart (`cpr-transfer`) is a lifecycle event the design had never considered.

One more cost the adapter design surfaces and does not argue away. A QEMU adapter needs extern "C" entry points, raw `MemoryRegion *` handles and adoption of a host pointer QEMU owns — all unsafe. The workspace's CI gate exists to keep the audit surface at *one* crate readable in one sitting, and its own failure text says adding a second is "a design decision, not a manifest edit". L2-Q takes that decision: `kayfabe-qemu-raw` becomes the second audited unsafe crate, and the ratchet moves from a constant to a named two-element list. The acceptance criterion the gate protects becomes "read two crates". That is the honest price and the design says so in those words.

8.11 The bench is a single serialised GPU

All hardware work happens on one rented GPU box, strictly serially: concurrent or mid-iocli activity wedges the GPU. The C artifact needs a **fresh VM boot per clean run** — and, per §3, that discipline turns out to be a bug rather than a precaution. Two cargo builds on one box corrupted a mutation run through the shared `~/cargo`, so the rule is one cargo per box.

The consequence for the reader: every hardware number in this document is $n = 1$ in the statistical sense and comes from one GPU model (GA106/GA107 class consumer Ampere) on one driver family. There is no second machine, no second GPU generation, and no continuous hardware testing.

8.12 The recurring failure mode: claims that were true *once*

This is the pattern the project keeps hitting, it has a name in the repository, and it is the single most useful thing for a reviewer to internalise.

Across three independent passes, roughly **twenty** instances were found of documentation asserting more than the code did — **several stating the literal opposite of their own code. Every one of them was true when written.** That is the point: this is not carelessness, it is entropy. [docs/design/testing_doctrine.md §6]

The instances are specific and they are not small:

- A design document named `memory_region_clear_global_locking()` as the mechanism for a decision. **That QEMU function had been deleted five years earlier.** The reference document written to price the floor now carries a `file:line` on every row for exactly this reason — and found two *more* live instances of the same decay while being written.
- A refactor plan cited a `userfaultfd` proof-of-concept file **that does not exist**. The mechanism was planned four times and never built; its demand-fault path is dead code that calls `stub_exit(139)` and never returns, so the fault address has been permanently zero since a commit early in the project's life. Nobody noticed because "planned" and "presumably done" read the same in a plan.
- **Two sections of one design document contradicted each other**, found only when the code between them was written.
- Two claims about the C's behaviour, cited from the C's own source comments, turned out to be **false when measured on the bench**. The rule extracted: citing a comment is citing a belief.
- A rustdoc described a default-to-GPU-0 guess *inside the one resolver whose entire discipline is "never guess"*. Another claimed "MISS=FAULT is preserved — never a silent skip" while its own body correctly skipped on both arms; the body was right and the sentence was wrong about the most-cited exception in the codebase.

The project's response is the right one: prefer a mechanism to a sentence. A rule stated only in prose has no reader. `HostHandle` carrying its `IsolateId` is the same rule with a compiler behind it. The house style for corrections is *strike visibly* — keep the original, struck, with the correction and its evidence — so a reader who remembers the old claim learns it was wrong rather than doubting their memory.

And the pattern is live right now, in this document's subject, and in this document. The first revision of this paper found fourteen instances (§10 rows 1–14). The second found six more, two of them in the first revision's own text and figures. **This revision found six more again, and three of them are in this paper** — including the headline claim it was built around, a table row that was already wrong when it was printed, and a count that has since moved in the wrong direction. Every single one, across all three passes, is a claim with *no gate behind it*.

Every claim that *does* have a gate is mechanically true today — the boundary greps, the generation-name grep, the ratchet at 37, the reached-count floor at 35, the “exactly one GPA call site” assertion, the E0639 compile-fail row that makes the ring gate structural, and now the bridge-exclusivity grep, *which fired on its first run*. Three revisions of this paper have now found roughly twenty-six ungated claims to be stale and **zero** gated ones. That is either a devastating footnote or the strongest possible evidence for the project's own thesis, and it is the latter.

★ **The sharpest form of it at this revision is not in the prose at all — it is in the instruments (§8.4).** A configuration file that was never read, a baseline that omitted the suite, a timeout scored as a kill and a hang that printed nothing are all the same species as a stale sentence: a description that used to be true of a thing. The difference is that those four described *the evidence*, which makes them the more expensive kind.

The strongest single instance is in §8.5 and it is worth restating here as a pattern rather than a fact about NVIDIA. A specification was cited as if it were the specification, when it was one version of a specification; 3 550 lines were built against it; and one question had been closed with an answer that is exactly backwards for the machine the code will first run on. The claim that survived that audit was the one marked [inferred] — because it knew it was second-hand. The claim that did not survive was marked **RESOLVED**.

8.13 Risks this document adds

- **Test code slightly exceeds implementation code, and the mocks are the only implementation of three ports.** The suite is therefore testing the core against a model of NVIDIA written by the same author as the core. `MockArch`'s deliberately non-NVIDIA encodings mitigate the *encoding* half of that; they do not mitigate the *semantics* half. If a belief about RM semantics is wrong, the suite will confirm it consistently.
- **The project is four days old and moves faster than its own documentation.** 125 commits in 94 hours, with several design documents rewritten mid-build. That is what produced the stale claims in §10, in a repository that has an explicit doctrine against stale claims. It has now produced three rounds of them: six after the first revision, two of those in this paper; six more after the second, *three* of those in this paper, including its headline. The rate is not slowing, and no gate covers prose.
- ★ **Test code now exceeds implementation code by 16%, and the gap is widening fastest exactly where nothing has been validated.** `tests/tests/rmrpc_bridge.rs` is four times the size of the crate it tests. The three-independent-transcription discipline that produced that ratio is the right response to §8.3, and it is still a suite whose every oracle was written by the author of the thing it checks. §8.3 is the measured demonstration that this is not a theoretical concern.
- **kayfabe-vmm-kvm defect #57 is a lock-invariant violation under race, and it is blocking by design.** That is the correct call. It is also the kind of defect that is cheap to live with and expensive to leave: an R1 violation that fires 1-in-6 under contention is exactly the shape that becomes a rare production hang.
- **The security model has been red-teamed at the logic layer only.** Memory safety, out-of-bounds and host-breakout are explicitly deferred to a post-L2 audit, on the sound argument that they are born at the `mmap/isolate/trap` surfaces and there is nothing to audit in pure logic. Those surfaces now partly exist (`kayfabe-linux-raw`, `kayfabe-vmm-kvm`), so the deferral is beginning to age.
- **The `Vmm` port's hypervisor-agnosticism is contracted and CI-gated but has exactly one adapter.** “A second backend costs one crate and zero trait changes” is a claim that only a second backend can settle.
- ★ **The bridge is built and it is not on the path to compute, which is a schedule risk rather than a design one.** §8.2: `MAP_MEMORY_DMA` is not on the wire, so finishing the object-model bridge to its last stage leaves the address table empty. The two real populate sources are in crates that are not built. Anyone reading “the bridge exists” as progress toward `cuCtxCreate` is reading it wrong, and the design document says so in its own words before anyone had to be told.
- ★ **Every mutation number this project has ever published described a population that included its own instruments,** and the gate that produces those numbers could have reported 100% from a broken suite

(§8.4). Both are fixed. Neither had any observable symptom before it was looked for, and the threshold is now deliberately unset until a clean campaign lands — which means **there is currently no mutation bar in force at all**.

- **One open item bears directly on a stated product requirement and has no proposed resolution.** §11-O7a (§8.5) may mean that “GPU restart works with no bolt-on” is not satisfiable at the bench’s driver version without version-conditional behaviour that the project’s own rules forbid in a logic crate. It cannot be closed from open source. Everything else in this list is a known quantity with a known fix; this one is not yet a known quantity.

9 How to read the code

9.1 Entry points

If you want to understand...	Read
the whole system, fastest	README.md, then ARCHITECTURE.md — but read §10 of this paper first, because both are stale in specific ways
what a “process” is and why the source of truth	crates/kayfabe-core/src/project.rs — the pure derivation crates/kayfabe-core/src/rmgraph.rs — the recounted graph and the parked-fact machinery
the transactional apply the forwarding entry points	crates/kayfabe-core/src/gpu.rs, Gpu::apply → Spine::apply crates/kayfabe-fwd/src/lib.rs — start at route_doorbell, then plan_doorbell; the ring gate itself now lives one crate over, at kayfabe-isolate VerbPlan::gated_doorbell, with its compile-fail pin in crates/kayfabe-isolate/tests/ui/
the faked GSP	crates/kayfabe-gsp/src/lib.rs (the module map and the five seams), then boot.rs — BootPhase×QueueState is the whole boot protocol; the test-side guest is tests/src/gspworld.rs
the GSP → core bridge	docs/design/gsp_core_bridge.md §2.7 <i>first</i> (it is the finding that changed the roadmap), then crates/kayfabe-rmipc/src/lib.rs translate and BridgeRefusal — the refusal enum is the honest map of what is not modelled
the concurrency design	crates/kayfabe-rt/src/lock.rs (ranks) then device.rs verb_op — the whole plan/execute/commit shape is one function
the unsafe surface	ls crates/kayfabe-linux-raw/src/*_unsafe.rs — that is the complete list, by construction
what a test is supposed to look like	tests/tests/l1_mean.rs (the arbiter) and tests/tests/miss_taxonomy.rs (the sharpest single property)

9.2 Conventions

- *_unsafe.rs — any file using the unsafe keyword is named this way, and the gate greps whole words so writing *about* the rule (unsafe_code, _unsafe.rs) never trips it.
- plan_* / commit_* — the two halves of the verb seam. plan_ runs under locks and issues nothing; commit_ runs after re-acquisition and re-validates.
- route_* / exec_* — the route/act split (structural rule R4): resolve identity against device-wide state, then act on one process’s state.
- Test names carry their provenance: t12_, t13_, t14_ are C-bug regressions; cb* are the regression matrix rows; b1_–b6_ are security boundary cases; i1–i4 are the isolation properties; the marker *Mutation-gate kill* in a doc comment names the mutant a test exists to kill.

9.3 Which design documents to read, in order

1. docs/design/testing_doctrine.md (321 lines) — read this *first*, even before the architecture. It is the shortest path to understanding what the project believes evidence is.
2. docs/design/core_state_and_consolidation.md — the reviewed per-crate state and the L1 hand-off contract.
3. docs/design/core_security_threat_model.md — the attacker model and the I1–I4 properties.
4. docs/design/l1_concurrency.md §12 and docs/design/l1_os_shell.md §14 — the contact logs (the two documents run to 10 265 lines between them). **Read these rather than the summaries**; they are where the design was found to be wrong, which is the part a summary cannot carry.

5. docs/reference/ — measured ground truth, kept out of the design documents so a wrong fact is corrected in one place. Note the version caveat in `rm_semantics_measured.md` §0 before quoting any number from it.

10 Claims corrected while writing this paper

Each of these is a claim that this paper checked against the tree and found stale or wrong. They are listed because the list is more useful than any individual correction: it is a measurement of the drift rate described in §8.12. None is a defect in the code; all are defects in descriptions of the code.

Rows 1–14 are from the first revision, at 0b78102; rows 15–20 from the second, at 6ce8599; rows 21–26 are new at b612a91. **Eleven of the first fourteen were fixed or superseded within four days**, and each row says which — that is the loop working, and it is the single most encouraging measurement in this paper. Row 9 is the instructive exception: it was fixed *in one document*, which then recorded four further documents still citing the retracted number and declared them out of scope.

Three of this revision's six rows are defects in this paper's own previous revision, and one of them is the sentence the previous revision was built around. That is the point rather than an embarrassment to be minimised: a whitepaper is a description of code and decays at the same rate as any other description of code. The claims that decay fastest are the ones stated most confidently, because confidence is what stops the next reader re-checking them.

	Claim, and where	What the tree says
1	<i>"handle_doorbell is the ONLY caller of</i> <code>RmBackend::ring_doorbell</code> " — <code>ARCHITECTURE.md</code> :112, <code>core_completeness_gate.md</code> :119, <code>core_state_and_consolidation.md</code> :174, <code>fwd/src/lib.rs</code> :15	False as stated. <code>ring_doorbell</code> has exactly one call site and it is inside <code>Worker::execute</code> , two hops away. <code>plan_doorbell</code> had <i>two</i> callers: <code>handle_doorbell</code> and <code>SharedDevice::doorbell</code> — and the L1 shell path, the one a real guest MMIO write takes, goes through the second and never enters <code>handle_doorbell</code> . The safety property survived; the cardinality claim did not . Superseded at 634b253: the gate moved into <code>VerbPlan::gated_doorbell</code> , the sole constructor of a <code>#[non_exhaustive]</code> variant, so the property no longer rests on a cardinality claim at all — it rests on E0639. <i>A correction that the code then made unnecessary is the best outcome on this list.</i>
2	<code>kayfabe-abi</code> is a <i>"stub (shape only)"</i> — <code>README.md</code> , <code>ARCHITECTURE.md</code> , and its own <code>Cargo.toml</code> description	False , and FIXED SINCE (e6ba19e). 7 455 lines of source, a working offline generator, 11 generated structs, a 13-method version-dispatch decode surface, 2 272 lines of oracle tests. All three sites now carry struck-through corrections. <i>Listed here because the correction is the evidence the house style works.</i>
3	<i>"the implementor is the L1 shell (kayfabe_rt::SharedDevice)"</i> — <code>ARCHITECTURE.md</code> :38--43, <code>kayfabe-vm/src/lib.rs</code> :816, <code>core/src/gpu.rs</code> :1012	No impl Device for SharedDevice exists. The only two implementors are test types that wrap an <code>Arc<SharedDevice></code> and forward. Still true at 6ce8599, but the descriptions are FIXED : both <code>ARCHITECTURE.md</code> and <code>kayfabe-vm/src/lib.rs</code> now state the absence in bold rather than implying the shell already does it.
4	<i>"283 tests"</i> — <code>README.md</code> , <code>ARCHITECTURE.md</code>	515 measured on 2026-07-27; 547 at this revision. FIXED SINCE (e6ba19e) — and fixed in the right way: both files now say <i>"in the 500s as of 2026-07-27"</i> rather than carrying a fresh exact number that would rot again in a week. <i>Deleting a number is a legitimate repair.</i>
5	<i>"four standing CI gates"</i> — <code>README.md</code>	Six jobs; twelve steps in the stable job alone. The list of four omitted the VMM-vocabulary gate, the GPA-accessor gate, three unsafe-containment asserts, the KVM reached-count floor, the fuzz-compile job, the slow job and the mutation job — and, since it was written, the generation-name gate. FIXED SINCE : the phrase no longer appears in <code>README.md</code> .
6	<i>"TSan (28 threaded tests, 0 races)"</i> — <code>README.md</code>	28 was the first campaign. The four TSan targets contain 68 <code>#[test]</code> functions at this revision (65 at the previous one). The <i>"0 races"</i> half is not disputed here — it was not re-run. FIXED SINCE : <code>README.md</code> now names 28 as <i>"the first campaign"</i> and says it has not been re-run.
7	<i>"the two measured-slow tests"</i> — <code>README.md</code>	Four to six, depending on how you count; the <code>g10_*</code> capped pair joined later. FIXED SINCE : the <code>README</code> now says membership <i>"has grown since it was measured"</i> and points at the <code>skip_slow!</code> call sites as the authority instead of quoting a count.

	Claim, and where	What the tree says
8	<i>"the conformance suite (23 files)"</i> — ARCHITECTURE.md	29 files in tests/tests/ at this revision, plus 6 under crates/*/tests/ (28 + 5 at the previous one). FIXED SINCE: the count was removed rather than updated.
9	core_mutation_gate.md's 91% threshold and its <i>Reproduce</i> block	Both described the pre-2026-07-27 four-path scope. FIXED SINCE (e6ba19e): the document now opens with a banner saying so. <i>But the correction propagated only one hop</i> — the same document records that the 92.44% and the 91% floor are "still cited as settled" by README.md, ARCHITECTURE.md, 11_architecture_summary.md and core_state_and_consolidation.md, and reports them as out of its own edit scope. A correction that names its own unpropagated instances is the honest form; it is still four live instances.
10	kayfabe-mmμ:walker's own doc: <i>"Also here (skeleton this milestone): the GMMU walk algorithm"</i> — mmμ/src/lib.rs:30	There is no algorithm: one trait, one enum, 41 lines, zero constructors, zero implementors — unchanged at 6ce8599 . The doc is FIXED SINCE: the module comment now carries a struck-through correction naming what it actually contains. <i>The code did not move; the claim did.</i>
11	kayfabe-gsp's doc: RPC decode <i>"against kayfabe-abi's generated message layouts"</i>	Was false; is now true. At the previous pin kayfabe-gsp did not depend on kayfabe-abi and nothing did. At 6ce8599 the manifest lists it and five of the eight modules use it. <i>A doc claim that was aspirational and became correct is still a drift instance — nothing gated it either way.</i>
12	<i>"Isolate/RmBackend/Vmm trait-only (mock-implemented)"</i> — ARCHITECTURE.md:31,56	True for Isolate/RmBackend. False for Vmm: crates/kayfabe-vmm-kvm is a real KVM adapter with a real memory plane. FIXED SINCE — ARCHITECTURE.md:31 now carries the struck-through original.
13	<i>"~14 months of quirk capture" in the C's architecture and ABI documents</i>	Git: 2026-05-24 to 2026-07-24. Two months, 739 commits.
14	The framing <i>"7B LLM at 63 tok/s"</i> as a headline C result	63.4 is a Mode-1 A/B endpoint. The audited parity figure is $0.97\times$ (66.0 guest vs 67.8 host). The underlying milestone note still records the pre-fix 20 tok/s and was never updated. All three numbers are real; they are different builds.
15	kayfabe-gsp is a stub — README.md:161, ARCHITECTURE.md:68, and its own Cargo.toml description, which still opens <i>"SKELETON — the faked GSP..."</i>	False. 3 550 lines, 8 modules, 24 dedicated tests, an independent 1 133-line guest model, and a composed run inside the arbiter suite. The two markdown rows at least carry an [unverified] hedge telling the reader to read lib.rs instead; the Cargo.toml description carries none. <i>This is the same shape as row 2, one crate later, four days after row 2 was written.</i>
16	mode2_gsp_port_plan.md §11-O7 marked RESOLVED: <i>"GSP_RUN_CPU_SEQUENCER is not implemented, do not emit it"</i>	Backwards for our bench, and the status has been withdrawn (7af23cb). That is 610's answer. At 580 it is fully implemented, dispatched, and first on the bootup-window allowlist. The whole plan's citations were taken from 610.43.02 while the bench runs 580.159.04; eight claims changed when the matching tree was vendored. See §8.5.
17	The plan's §1.3: <i>"the guest declares its own geometry"</i> , presented as the highest-leverage design choice	610 only. At 580 MESSAGE_QUEUE_INIT_ARGUMENTS has four fields, not nine, and the queue geometry is compile-time. On the bench nothing is negotiated, so the argument the design leaned on does not apply to the machine it will first run on.
18	121 bare ogkm: citations in the crates' doc comments, 96 of them in kayfabe-gsp	Every one is a 610 path with a 610 line number , and several name files that do not exist at 580 — the whole msgq library moved directories between the tags. The version-tagging rule that fixes the class is normative in the plan and not applied to the code ; the CI grep that would prevent the recurrence is specified and not written.
19	This paper's own previous revision: <i>"kayfabe-abi and kayfabe-gsp are disconnected islands"</i>	Half false at 6ce8599. abi has a consumer now (gsp); gsp still has none outside tests/. Re-checked against the manifests, not inherited from the caption.
20	This paper's own previous revision: Figure 2, <i>"all 65 [dependencies] edges are shown"</i>	The manifests had 66. The edge tests → rt was omitted, in a figure whose stated method is <i>"derived from the manifests"</i> . A drawn matrix is exactly as ungated as a sentence.

	Claim, and where	What the tree says
21	★★ This paper's own previous revision, its headline: <i>"RpcCommand has zero references anywhere in the tree outside the crate that defines it ... the critical path is the bridge, not the crate"</i>	False at b612a91. <code>kayfabe-rmrcp</code> (76a0077-3a1704f) names it in three files and is a production crate depending on both <code>gsp</code> and <code>core</code> ; a CI grep now enforces that it is the <i>only</i> crate permitted to. <i>The claim was true when written and was invalidated by the work it identified as necessary</i> — which is the best possible outcome for a criticism, and still a stale sentence four days later. The successor claim, stated so it can be checked the same way: nothing but <code>tests/Cargo.toml</code> names <code>kayfabe-rmrcp</code> .
22	This paper's own previous revision: <i>"Checked-in in proptest failures: 0 — find for proptest-regressions: nothing exists"</i>	Wrong when printed. Four <code>*.proptest-regressions</code> files existed at 6ce8599 holding 18 seeds; there are 20 now. The row's stated method was a <code>find</code> that would have returned them. <i>This is the only correction on this list that is a measurement error rather than decay</i> , and it is the one that most deserves to be here: the method section claimed a command was run, and the number it reports is not the number that command produces.
23	This paper's own previous revision: the GSP-S1 row, <i>"Nothing. It is a design-doc invariant with no code behind it ... grep finds the variant nowhere"</i> , and the risk-list item calling it <i>"the highest-stakes sentence it has"</i>	Fixed since (250b78c). <code>GspFault::ElementCountOutOfRange</code> exists at the write site, unconditionally, plus a receive-side range check and a derivation cross-check. Noted here rather than silently updated, because this paper named the gap and the gap closed — that is the loop working on the paper's own output.
24	<code>gsp_core_bridge.md</code> §4.3: <i>"known-and-inert versus unknown — there is no third state"</i> ; §5.2's B2/B4 non-vacuity arms; §6's Memory-alloc row; §7.1's assumption that <code>SET_PAGE_DIRECTORY</code> carries the PDB	All corrected by the build, in the commit that hit them. There are at least four dispositions, not two. Three non-vacuity arms were unbuildable at the stage that specified them and one was <i>wrong</i> (refusing a second PDB requires state a stateless bridge cannot have, and <code>RmGraph</code> had already decided the other way with an argument). The Memory row is unbuildable twice over — the address fields are <code>[OUT]</code> in the <code>guest</code> → <code>GSP</code> direction, and its only consumer is driven by an event with no producer. §7.1 is settled against the design: see §8.2.
25	<code>mutants.toml</code> 's own header, and every "excluding scaffolding" figure this project has published, including one in this paper's previous revision	★★ The file was never read (b64a3f5). It lived at the workspace root; <code>cargo-mutants</code> reads <code>.cargo/mutants.toml</code> . Every exclusion was inert and every scaffolding-excluded score was a hand subtraction. Proven, not inferred: <code>--list</code> targets 3760 → 2667, scaffolding mutants 361 → 0. <i>An inert configuration file is the same failure as a stale sentence, with worse consequences.</i>
26	<i>"121 bare ogkm: citations, and the version-tagging rule is normative"</i> — §10 row 18, at the previous revision	The rule is being followed by new code and the backlog grew. 56 tagged citations now exist where there were none; bare untagged ones went 121 → 159. The CI grep that would stop it is still specified and not written, and it is now the difference between a rule and a wish.

10.1 What this paper could not verify

- **Any claim requiring a build or a run.** No `cargo` command was executed while writing this, at any of the three revisions. Test counts, mutation scores, TSan results, timings and the `arm64` run are all read from commit messages, workflow files and design documents, and are attributed as such. The 651 figure in particular is a recorded measurement from b612a91, not one taken here.
- ★ **The current mutation score, deliberately and by the project's own rule.** The 2898-mutant campaign completed, but against a tree modified underneath it, and under a configuration that has since changed the target population from 2898 to 2667. Its 95.52% is reported in §7.6 as an indication with that label attached and **must not be quoted as a measurement**. The clean campaign is in flight and its number was not available when this was written. There is consequently *no* defensible threshold to state, and this document deliberately states none.
- **Anything about the four rows of the survivor taxonomy beyond its own report.** "8 provably equivalent, 22 dead public API, 17 scaffolding, 68 real logic gaps" is read from 92ae2e1's commit message [unverified]; the individual arguments were not re-derived here, though a sample of three equivalence arguments in 0ec15da was read and each is argued at the site rather than asserted.
- **Anything about NVIDIA's 580 driver that this paper did not read itself.** The §8.5 claims about 580-vs-

610 are taken from docs/design/gsp_580_correction_brief.md and mode2_gsp_port_plan.md §14, which state their own limit: only 580.159.04 and 610.43.02 are vendored and were read directly. Seven further driver tags used to narrow the element-layout break interval are **relayed from another agent’s probe and were read by nobody in this chain [unverified]**. The predicate the project derived (major >= 610) is safe under either reading because the 610 boundary is the directly-verified one; the sentence “580, 590 and 595 are all on the 48-byte side” is not.

- **Whether the 580 resume path is ever taken.** §11-O7a is open, it is the sharpest risk in §8.5, and the evidence that would settle it — the contents of a sequencer buffer — is in closed GSP-RM firmware. Nothing here narrowed it.
- **“TSan, 0 races.”** Not re-run; only the target list and the test count in those targets were checked.
- **The relationship between the C’s 2026-06-16 bare-metal result and the 2026-07-25 one-process-per-lifetime finding.** The memory note flags this itself: the finding was not cross-checked against the bare-metal box, and warns against assuming the C regressed.
- **Whether Mode 2 ever produced any display output.** No positive evidence was found, only the plan. Absence of evidence.
- **The #57 defect’s exact reproduction rate.** “~1 in 6 under race” is quoted from crate_maturity_map.md; it was not reproduced here.

A Measurement method

Every number in this paper that is not attributed to a document was produced by one of the following, run against /workspace/nvkvms at b612a91 and /workspace/nvidia-gpu-passthrough at 2899a74. No compilation, no test execution and no network access was involved. *The measurements were taken from a clean extraction of the pinned commit* (git archive b612a91), not from a working tree, because the working tree at the time again carried unrelated in-flight edits from concurrent work — two files, both belonging to another writer. The previous revision’s numbers were re-taken from a clean extraction of 6ce8599 in the same way, which is how §10 row 22 was found.

Quantity	Command
implementation lines	find crates -path '*/src/*' -name '*.rs' xargs wc -l
test lines	the same over tests, crates/*/tests, fuzz
per-crate lines	the same, per crates/<name>
#[test] sites	grep -rc '#[test]' over workspace members, excluding the detached crates/kayfabe-abi/gen and the trybuild ui fixtures
unsafe relaxations	grep -rhoE 'unsafe ({ fn impl trait })' crates/kayfabe-linux-raw/src/*_unsafe.rs wc -l — the same expression the CI ratchet uses
dependency edges	every Cargo.toml read in full, cross-checked against Cargo.lock
commit counts and dates	git rev-list --count HEAD, git log --format=%ci
document staleness	git log -1 --format=%ci per file, compared against the commit that invalidated the claim

The five figures are TikZ, drawn from the measurements above and from a function-by-function read of the call chains they depict; they are vector, not traced from an image. Figure 2 in particular is generated from the manifest edge list rather than from any design document, which is why it disagrees with the hexagon picture about which crates are “ports”.

Two method rules carried forward, both prompted by the figures being where the drift was worst. Figure 3 names *symbols* rather than file:line pins, as the repository itself did to its own 105 pins in 3fd1455. And Figure 2’s edge list is re-derived from scratch every revision rather than edited — it was re-derived by script this time, which is how the mocks → abi edge and the new topological ordering were picked up without being looked for.

One method rule added at this revision. Every claim carried over from the previous revision was re-measured rather than trusted, *including the ones this paper itself originated*. That is the only reason rows 21–23 exist: two of them are claims this document asserted with no hedge, and one of them was already false when it was printed. The rule that generalises is the one the project applies to its own instruments — **re-derive, do not restate**, because a restated number and a measured number are indistinguishable on the page.

One closing note on how to use this paper. If you are here to find what is wrong with kayfabe, the highest-yield reading order is: §8.2 (the bridge is built and it is not on the path to compute), §8.4 (the evidence machinery was wrong in four ways, all of them flattering), §8.3 (an independent oracle that was independently written and identically wrong), §8.5 (the open resume handoff, which still has no proposed resolution), §8.10 (the packaging cost the floor decision did not buy its way out of), §7's final subsection, the two contact logs named in §9.3, and then the code — starting with `VerbPlan::gated_doorbell`, `Spine::apply` and `rmrpc::translate`, which between them contain most of the load-bearing decisions. Do not start with `README.md`; §10 lists six things wrong with it (one of them since fixed), and it will cost you an hour.

And one note on how to read its confidence. Each of the three revisions of this document has found its own predecessor wrong in two or three places, and at this revision the falsified claim was the one the previous revision led with. Treat every unhedged sentence here the same way — including this one.